

Instituto Tecnológico de Costa Rica
Escuela de Ingeniería Electrónica
Unidad Desconcentrada de Ingeniería Electrónica - San Carlos

Taller de Diseño Digital

Guía y Proyectos

Profesor
Dr. Luis Gerardo León Vega

24 de mayo de 2026

1. Introducción	7
2. Referencia para Verilog	9
2.1. Estructura básica de un módulo	9
2.2. Tipos de datos	10
2.3. Operadores	10
2.4. Representación numérica	11
2.5. Asignaciones	11
2.6. always combinacional	12
2.7. always secuencial	13
2.8. if / case	14
2.9. Instanciación	15
2.10. generate (repetición)	16
2.11. FSM (plantilla rápida)	16
2.12. Testbench mínimo	18
2.13. System tasks útiles	18
2.14. Buenas prácticas (RTL)	19
2.15. Errores comunes	19
3. Flujo de Síntesis para FPGA y ASIC	21
3.1. Introducción	21
3.2. Flujo general de diseño digital	21
3.3. Descripción del diseño	21
3.4. Simulación y verificación	22
3.5. Síntesis lógica	23
3.6. Implementación en FPGA	23
3.7. Flujo de diseño para FPGA	23
3.8. Flujo de diseño para ASIC	23

3.9. Comparación entre FPGA y ASIC	24
3.10. Resumen	24
4. Implementación de Lógica Combinacional	27
4.1. Introducción	27
4.2. Implementación de lógica combinacional en Verilog	28
4.3. Asignaciones continuas	28
4.4. Uso de bloques <code>always</code>	29
4.5. Multiplexores	30
4.6. Uso de <code>case</code>	31
4.7. Buenas prácticas en lógica combinacional	32
4.8. Resumen	32
4.9. Ejercicios	32
4.10. Laboratorio	33
5. Bancos de Prueba	35
5.1. Introducción	35
5.2. Estructura de un banco de pruebas	35
5.3. Ejemplo de diseño a verificar	36
5.4. Creación de un banco de pruebas	36
5.5. Tipos de señales en un testbench	37
5.6. Control del tiempo en simulación	37
5.7. Visualización de señales	38
5.8. Generación automática de formas de onda	38
5.9. Ejemplo completo de banco de pruebas	39
5.10. Buenas prácticas	39
5.11. Resumen	40
5.12. Ejercicios	40
5.13. Laboratorio	40
6. Implementación de Lógica Secuencial	41
6.1. Introducción	41
6.2. Señales de reloj	42
6.3. Flip-Flop tipo D	42
6.4. Registro	43
6.5. Registro con habilitación	43
6.6. Contadores	44
6.7. Reset	44
6.8. Máquinas de estados finitos	45
6.9. Buenas prácticas	45
6.10. Resumen	46
6.11. Ejercicios	46
6.12. Laboratorio	46

7. Consideraciones de Entrada/Salida	47
7.1. Introducción	47
7.2. Señales asíncronas	47
7.3. Metastabilidad	48
7.4. Sincronización de señales	48
7.5. Debouncing de botones	49
7.6. Debouncer basado en contador	49
7.7. Sincronización de señales de reset	50
7.8. Constraints de I/O	50
7.9. Niveles lógicos	51
7.10. Resistencias pull-up y pull-down	51
7.11. Resumen	51
7.12. Ejercicios	52
7.13. Laboratorio	52
8. Microarquitecturas	53
8.1. Introducción	53
8.2. Métricas de desempeño	54
8.3. Microarquitectura uniciclo	54
8.4. Microarquitectura multiciclo	55
8.5. Microarquitectura segmentada	56
8.6. Comparación de microarquitecturas	56
8.7. Compromisos de diseño	57
8.8. Resumen	57
8.9. Laboratorio	57
9. Diseño Digital Multinivel	59
9.1. Introducción	59
9.2. Concepto de jerarquía	59
9.3. Principios de diseño	60
9.4. Flujo de diseño multinivel	61
9.5. Buenas prácticas de diseño	66
10. Scripting y Automatización	67
10.1. Introducción	67
10.2. Scripting con Bash	69
10.3. Scripting con TCL	72
10.4. Procesamiento de datos con Awk	75
10.5. Flujo automatizado	77
10.6. Buenas prácticas	77
10.7. Resumen	77

11. Control de Versiones con Git	79
11.1. Git	79
11.2. ¿Por qué usar Git en diseño digital?	80
11.3. Modelo mental básico de Git	80
11.4. Primer flujo de trabajo con Git	81
11.5. Comandos fundamentales	83
11.6. Ramas	84
11.7. Conflictos de fusión	85
11.8. Repositorios remotos	85
11.9. Archivos que no deben versionarse	86
11.10. Buenas prácticas generales con Git	87
11.11. Errores comunes	91
11.12. Resumen	92
12. Protocolos de Comunicación	93
12.1. Comunicación UART	93
12.2. Comunicación SPI	95
12.3. Comunicación I2C	98
12.4. Comunicación USB	102
12.5. Protocolo AXI	104
13. Conceptos de Diseño en FPGA	111
13.1. Introducción	111
13.2. Síntesis de Memorias en FPGA	111
13.3. Síntesis de DSPs en FPGA	115
14. Introducción al Lenguaje C para Sistemas Embebidos	117
14.1. Introducción	117
14.2. Flujo de compilación y ejecución	117
14.3. Constructs básicos del lenguaje	118
14.4. Flujo de programa	119
14.5. Modelo de memoria	120
14.6. Estructuras y memoria	121
14.7. Punteros	124
14.8. Punteros a funciones	125
14.9. Laboratorio: Sistema embebido interactivo con Arduino	126

CAPÍTULO 1

Introducción

En este documento, se resumirá la guía general del curso de Taller de Diseño Digital, incluida una referencia rápida para Verilog / SystemVerilog, microarquitecturas de diseño digital, como las vistas en el curso de Diseño de Sistemas Digitales y los laboratorios prácticos a ejecutarse a lo largo del taller.

2.1. Estructura básica de un módulo

Los módulos son bloques de hardware o conjuntos de circuitos que representan una lógica digital en particular. Por ejemplo, un módulo puede ser desde una compuerta hasta un sumador completo de N bits.

```
module and2 (  
    input wire a,  
    input wire b,  
    output wire y  
);  
    assign y = a & b;  
endmodule
```

Con parámetros:

Los módulos pueden parametrizarse para hacerlos más genéricos. Por ejemplo, podemos hacer un módulo sumador que tenga un parámetro ajustable N para representar la cantidad de bits de los operandos.

```
module sum #(  
    parameter N = 8  
)(  
    input wire [N-1:0] a,  
    input wire [N-1:0] b,  
    output wire [N-1:0] y  
);
```

```
assign y = a + b;
endmodule
```

2.2. Tipos de datos

Nets (conexiones): wire, tri (buses triestado).

Variables (procedural): reg, integer, genvar (para bloques generate).

```
wire      a;
wire [7:0] bus;
reg       q;
reg [3:0] cnt;
integer   i;
```

Nota 1: reg *no implica* flip-flop por sí mismo; depende del always.

Nota 2: En SystemVerilog, se abstrae el tipo wire con el tipo reg en un nuevo tipo llamado logic, que resulta más conveniente.

2.3. Operadores

Lógicos vs bit a bit

- Lógicos: &&, ||, ! (devuelven 1 bit).
- Bit a bit: &, |, ^, ~.

Desplazamiento

```
a << 2;  a >> 1;  // logico
a <<< 1;  a >>> 1; // aritmetico
```

Concatenación y replicación

```
{a, b}      // concatena
{4{a}}      // replica 4 veces
```

2.4. Representación numérica

Formato: <ancho>'<base><valor>

```
8'b10101010 // binario
8'hFF        // hexadecimal
8'd255       // decimal
8'o77        // octal
```

2.5. Asignaciones

Continua (combinacional)

Funciona similar a una función booleana.

```
assign y = (a & b) | c;
```

Solo para nets (típicamente wire).

Procedurales (en always)

Funcionan como fragmentos de código. Se pueden realizar funciones booleanas más complejas, con múltiples salidas que se interrelacionan entre sí. Inclusive, se pueden hacer composiciones de funciones.

Bloqueante = (combinacional) vs **No bloqueante** <= (secuencial).

En el contexto de Verilog, una asignación bloqueante implica que el sintetizador debe respetar el orden de las asignaciones.

Una asignación no bloqueante le da libertad al sintetizador de calendarizar la ejecución de las asignaciones en el mismo ciclo de reloj pero sin importar el orden

```
// combinacional
always @(*) begin
    y = a ^ b;
end

// secuencial
always @(posedge clk) begin
    q <= d;
end
```

Regla práctica: usa = en combinacional y <= en secuencial.

Un caso de ejemplo:

```
// combinacional
always @(*) begin
    y1 = a ^ b;
    y2 = y1 | c;
end

assign d = y2;
```

En este contexto, para ejecutar $y2$, se requiere ejecutar $y1$. Entonces, $y2$ se ejecuta hasta tener $y1$ listo. Visto como un circuito, es como conectar el circuito de $y2$ después del circuito de $y1$, como se ve en Figura 2.1.

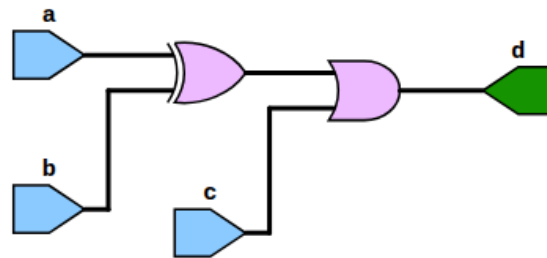


Figura 2.1: Asignación bloqueante

```
// secuencial
always @(posedge clk) begin
    y1 <= a ^ b;
    y2 <= y1 | c;
end

assign d = y2;
```

En este caso, se observa que las operaciones se separan a través de un flip-flop, por lo cual, el dato $y1$ se transmite y se termina de calcular $y2$ hasta en un ciclo de reloj siguiente, como se ve en Figura 2.2.

2.6. always combinacional

Se define lógica combinacional (sin reloj), lo que permite la implementación de arquitecturas puramente combinacionales o uniciclo.

Patrón robusto: valores por defecto + cubrir todos los caminos (evita latches).

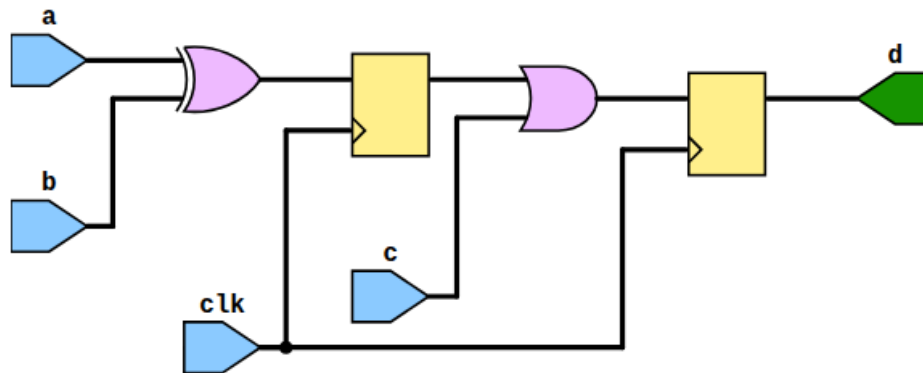


Figura 2.2: Asignación no bloqueante

```
always @(*) begin
    y = 1'b0;           // default
    if (en)
        y = a & b;
end
```

Nota: En SystemVerilog, se recomienda el uso de `always_comb`:

```
always_comb begin
    y = 1'b0;           // default
    if (en)
        y = a & b;
end
```

2.7. always secuencial

Se define lógica secuencial (con reloj), lo que permite la implementación de microarquitecturas segmentadas o multiciclo.

Flip-flop D

Este fragmento de código tipifica al flip-flop tipo D. Eso es útil tenerlo en cuenta cuando uno quiere hacer un *pipeline*.

```
always @(posedge clk) begin
    q <= d;
end
```

Reset síncrono vs asíncrono

Cuando se tienen microarquitecturas secuenciales, es necesario un punto de reinicio, por lo que hay dos posibles implementaciones:

```
// reset sincronico
always @(posedge clk) begin
    if (rst) q <= 1'b0;
    else    q <= d;
end

// reset asincronico
always @(posedge clk or posedge rst) begin
    if (rst) q <= 1'b0;
    else    q <= d;
end
```

Nota: En SystemVerilog, se recomienda el uso de `always_ff`:

```
always_ff @(posedge clk) begin
    if (en)
        y <= a & b;
    else
        y <= 1b'0;
end
```

2.8. if / case

Los condicionales o *cases* nos ayudan a implementar lógica condicional. En el contexto de los *cases*, pueden ayudarnos a implementar muxes que tengan una variable de condición o conmutación (*select*).

if-else

En este caso, se puede ver que se depende de una variable booleana, que es implementable mediante un mux de dos entradas.

```
always @(*) begin
    if (sel) y = a;
    else    y = b;
end
```

En este caso, se puede ver que se depende de una variable de dos bits, que es implementable mediante un mux de 4x1.

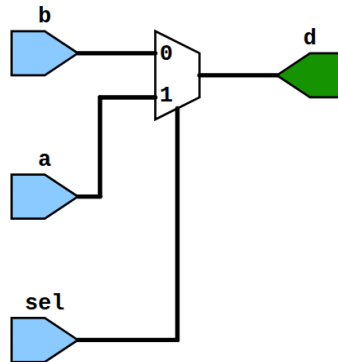


Figura 2.3: Ejemplificación de cómo se implementa un mux

case (incluye default)

```
always @(*) begin
  case (sel)
    2'b00: y = a;
    2'b01: y = b;
    2'b10: y = c;
    default: y = 0;
  endcase
end
```

2.9. Instanciación

La instanciación es cuando se coloca un módulo dentro de otro módulo externo. La analogía es, por ejemplo, crear un bloque que está hecho de bloques más pequeños. Esos bloques más pequeños se insertan mediante la instanciación en un bloque más externo o más grande.

Posicional

La posicional se basa en seguir el orden estricto en el que se colocaron los puertos del módulo. Es poco recomendado porque es menos mantenible.

```
and2 u1 (a, b, y);
```

Nombrada (recomendada)

La nombrada se basa en asociar los puertos del módulo a instanciar con las señales del módulo exterior. Es recomendada ya que evita problemas de mantenibilidad al no seguir un orden estricto.

```
and2 u1 (
  .a(a),
  .b(b),
  .y(y)
);
```

2.10. generate (repetición)

El bloque *generate* ayuda a repetir el código dentro del bloque. Es útil cuando se requiere metaprogramación o replicar el hardware usando parámetros.

```
module generate_ejemplo #(N = 8)
(
  input reg[(N-1):0] a,
  input reg[(N-1):0] b,
  output reg[(N-1):0] y
);
  genvar i;
  generate
  for (i=0; i<N; i=i+1) begin : GEN
    assign y[i] = a[i] & b[i];
  end
  endgenerate
endmodule
```

2.11. FSM (plantilla rápida)

Estados con localparam:

```
// Declaracion de los estados y de los registros

localparam IDLE=2'b00, LOAD=2'b01, DONE=2'b10;
reg [1:0] state, next_state;

// Registro de estado:

always @(posedge clk or posedge rst) begin
  if (rst) state <= IDLE;
  else    state <= next_state;
end

// Logica de estado proximo:

always @(*) begin
  next_state = state; // default
```

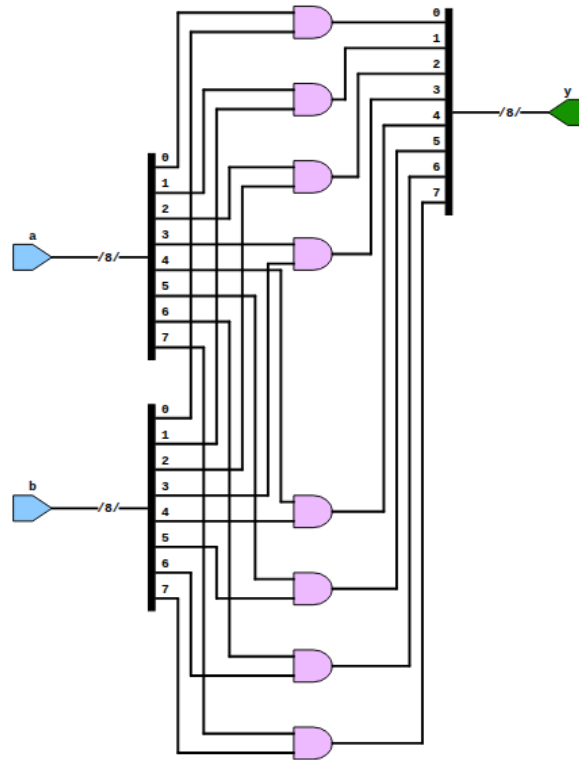



Figura 2.4: Ejemplo de generate

```

case (state)
  IDLE: next_state = LOAD;
  LOAD: next_state = DONE;
  DONE: next_state = IDLE;
  default: next_state = IDLE;
endcase
end

```

En SystemVerilog, la lógica puede cambiar a usar tipos introducidos por este lenguaje:

```

typedef enum logic[1:0] {
  IDLE=2'b00, LOAD=2'b01, DONE=2'b10
} state_t;

// Registro de estado:
state_t next_state, state;
always_ff @(posedge clk or posedge rst)
begin
  if (rst) state <= IDLE;

```

```

    else state <= next_state;
end

// Logica de estado proximo:
always_comb
begin
    next_state = state; // default
    case (state)
        IDLE: next_state = LOAD;
        LOAD: next_state = DONE;
        DONE: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end
end

```

2.12. Testbench mínimo

Los *testbench* son códigos que permiten probar la unidad de hardware desarrollada (UUT) mediante el establecimiento de señales. Esto permite ejercitar todas las combinaciones requeridas y poder observar señales luego.

```

// Modulo de testbench no lleva puertos
module tb;
    reg clk, rst;
    wire [3:0] q;

    // Instanciar la unidad en prueba
    contador uut (.clk(clk), .rst(rst), .q(q));

    // Crear un reloj
    always #5 clk = ~clk;

    // Setea el entorno de simulacion y coloca las senales
    initial begin
        clk = 0; rst = 1;
        #12 rst = 0;
        #200 $finish;
    end
endmodule

```

2.13. System tasks útiles

```

// Imprimir una vez
$display("q=%0d", q);

```

```
// Imprimir cada vez que haya un cambio  
$monitor("t=%0t clk=%b q=%b", $time, clk, q);  
// Detener la simulacion (checkpoint)  
$stop;  
// Finalizar la simulacion  
$finish;
```

2.14. Buenas prácticas (RTL)

- Separa combinacional (`always @(*)`) de secuencial (`posedge clk`).
- Evita latches: defaults + cubrir todos los caminos (`if/case` con `default`).
- No mezcles `=` y `<=` en el mismo bloque.
- Prefiere instanciación nombrada (`.puerto(senal)`).
- Parametriza anchos (`parameter`).

2.15. Errores comunes

- **Latch accidental:** no asignar en todos los caminos.
- **Sensibilidad incorrecta:** usar `@(*)` para combinacional.
- **case sin default:** comportamiento no definido ante valores no cubiertos.
- **Confundir `&&` con `&`:** lógico vs bit a bit.

3.1. Introducción

El diseño de sistemas digitales modernos se realiza principalmente mediante lenguajes de descripción de hardware (HDL), como Verilog o VHDL. Estos lenguajes permiten describir el comportamiento y la estructura de circuitos digitales a distintos niveles de abstracción.

Para transformar una descripción en HDL en hardware físico es necesario seguir una serie de etapas conocidas como el **flujo de diseño digital**. Este flujo incluye procesos de verificación, síntesis y optimización antes de generar el hardware final.

Dependiendo del tipo de dispositivo objetivo, el flujo de diseño puede dividirse en dos categorías principales: FPGA y ASIC.

Ambos flujos comparten varias etapas iniciales, aunque difieren significativamente en las fases finales de implementación.

3.2. Flujo general de diseño digital

En términos generales, el proceso de desarrollo de hardware digital se ilustra en la Figura 3.1, mostrando una representación simplificada de este flujo.

3.3. Descripción del diseño

El primer paso consiste en describir el circuito digital mediante un lenguaje HDL. En esta etapa se definen los módulos, señales, interfaces y comportamiento del sistema.

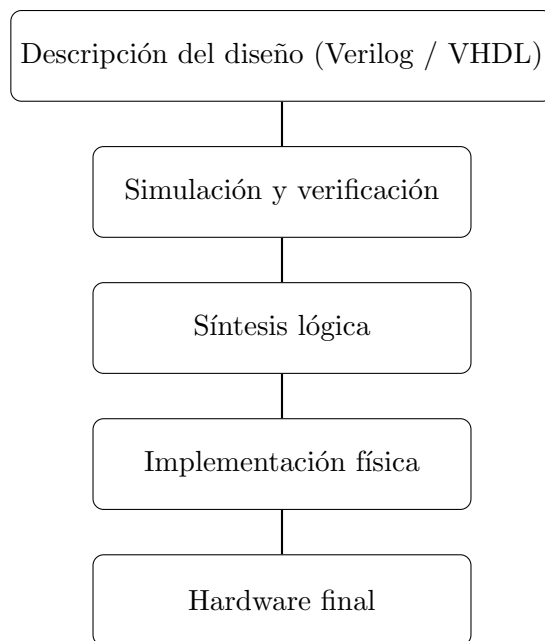


Figura 3.1: Flujo general de diseño digital.

El diseñador puede trabajar en diferentes niveles de abstracción, describiendo los circuitos por su comportamiento, como RTL o nivel estructural (como esquemáticos).

En la mayoría de los diseños modernos se utiliza el nivel RTL, que describe las transferencias de datos entre registros sincronizados por reloj.

3.4. Simulación y verificación

Antes de sintetizar el diseño es fundamental verificar su comportamiento mediante simulación. Para ello se utilizan bancos de prueba que generan estímulos y permiten observar las salidas del sistema.

La simulación permite detectar errores funcionales antes de proceder con la implementación física.

Las herramientas de simulación más utilizadas incluyen:

- ModelSim
- QuestaSim
- Verilator
- Icarus Verilog

3.5. Síntesis lógica

La síntesis es el proceso mediante el cual una descripción RTL se transforma en una red de puertas lógicas equivalentes.

Durante esta etapa, el sintetizador interpreta la descripción HDL, optimiza el circuito y genera una red lógica equivalente. El resultado de la síntesis suele representarse como una **netlist**, que describe el circuito en términos de componentes lógicos básicos.

Herramientas comunes de síntesis incluyen:

- Vivado Synthesis
- Quartus Synthesis
- Synopsys Design Compiler

3.6. Implementación en FPGA

En el caso de las FPGA, después de la síntesis se realiza un proceso conocido como **implementación**, que incluye:

- **Mapping**: asignación de la lógica a recursos de la FPGA (LUTs, flip-flops, etc.)
- **Placement**: ubicación física de los bloques en el dispositivo
- **Routing**: conexión de los bloques mediante la red de interconexión

Una vez completadas estas etapas, se genera el **bitstream**, que es el archivo utilizado para configurar la FPGA.

3.7. Flujo de diseño para FPGA

El flujo completo para FPGA puede resumirse como en la Figura 3.2.

3.8. Flujo de diseño para ASIC

El flujo de diseño para ASIC es más complejo, ya que el circuito se fabrica físicamente en silicio. Después de la síntesis lógica, el proceso incluye floorplanning, placement, clock tree synthesis, routing y verificación física. Finalmente, se generan los archivos necesarios para la fabricación del chip.

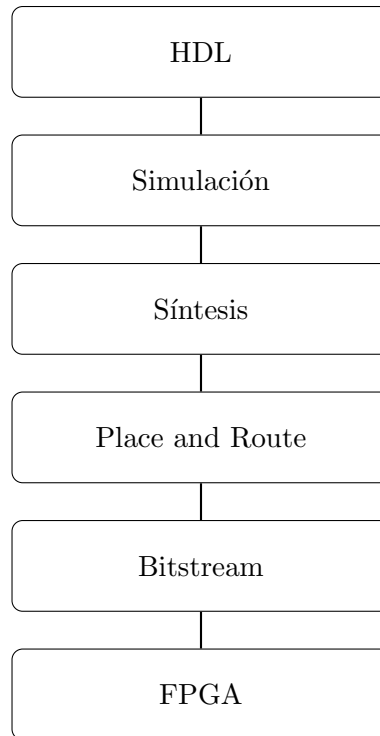


Figura 3.2: Flujo de diseño para FPGA.

3.9. Comparación entre FPGA y ASIC

Característica	FPGA	ASIC
Costo inicial	Bajo	Muy alto
Flexibilidad	Reconfigurable	Fijo
Tiempo de desarrollo	Corto	Largo
Rendimiento	Medio	Alto
Consumo de energía	Mayor	Menor

Las FPGA se utilizan principalmente para prototipado, investigación y sistemas reconfigurables, mientras que los ASIC se emplean en producción masiva.

3.10. Resumen

El flujo de diseño digital transforma una descripción en HDL en hardware físico mediante una serie de etapas que incluyen verificación, síntesis y optimización.

Aunque los flujos de FPGA y ASIC comparten etapas iniciales similares, difieren en las fases finales de implementación. Comprender estas etapas es fundamental para diseñar sistemas digitales eficientes y correctos.

4.1. Introducción

La lógica combinacional constituye uno de los bloques fundamentales en el diseño de sistemas digitales. En este tipo de circuitos, las salidas dependen únicamente de los valores actuales de las entradas y no del historial previo del sistema. Esto contrasta con la lógica secuencial, donde las salidas pueden depender de estados almacenados.

Formalmente, un circuito combinacional puede representarse como una función booleana:

$$Y = f(X_1, X_2, \dots, X_n)$$

donde:

- $X_1, X_2, \dots, X_n$ son las entradas
- Y es la salida

Los circuitos combinacionales pueden implementarse mediante compuertas lógicas básicas como:

- AND
- OR
- NOT
- XOR

Ejemplos clásicos de circuitos combinacionales incluyen:

- multiplexores
- decodificadores
- comparadores
- sumadores
- unidades aritméticas y lógicas (ALU)

En hardware programable como las FPGA, estos circuitos se describen utilizando lenguajes de descripción de hardware (HDL) como Verilog.

4.2. Implementación de lógica combinacional en Verilog

En Verilog, la lógica combinacional puede describirse de dos maneras principales:

1. Asignaciones continuas (`assign`)
2. Bloques procedurales (`always`)

Ambos enfoques pueden sintetizarse al mismo hardware cuando se utilizan correctamente.

4.3. Asignaciones continuas

Las asignaciones continuas permiten describir directamente funciones booleanas mediante operadores lógicos y aritméticos. Estas asignaciones modelan conexiones cableadas entre señales.

La sintaxis general es:

```
assign salida = expresion_logica;
```

La expresión lógica se evalúa continuamente y la salida se actualiza cada vez que cambian sus entradas.

Ejemplo: compuerta AND

La función lógica es:

$$Y = A \cdot B$$

Implementación en Verilog:

```
module and_gate (  
    input wire A,  
    input wire B,  
    output wire Y  
);
```

```
assign Y = A & B;

endmodule
```

La descripción en Verilog del módulo anterior corresponde directamente a una compuerta AND de dos entradas.

Ejemplo: función booleana

Considere la función:

$$F = AB + \overline{C}$$

Implementación en Verilog:

```
module logic_function (
    input wire A,
    input wire B,
    input wire C,
    output wire F
);

assign F = (A & B) | (~C);

endmodule
```

Los operadores de Verilog se corresponden directamente con las puertas lógicas.

Operador	Puerta lógica
&	AND
	OR
~	NOT
^	XOR

4.4. Uso de bloques always

Otra forma de describir lógica combinacional es mediante bloques **always**. En este caso se utilizan asignaciones procedurales.

Para garantizar que el hardware sintetizado sea combinacional se utiliza:

```
always @(*)
```

o en SystemVerilog:

```
always_comb
```

Esto indica que el bloque se ejecuta cada vez que cambia cualquiera de las señales utilizadas en él.

Ejemplo equivalente usando always

Considere nuevamente la función:

$$F = AB + \overline{C}$$

Implementación con always:

```
module logic_function_always (
    input wire A,
    input wire B,
    input wire C,
    output reg F
);

always @(*) begin
    F = (A & B) | (~C);
end

endmodule
```

Obsérvese que:

- la salida debe declararse como **reg**
- la asignación dentro del bloque se realiza mediante **=**

El hardware sintetizado es equivalente al ejemplo anterior.

4.5. Multiplexores

Los multiplexores son bloques combinacionales que seleccionan una de varias entradas según una señal de control.

Para un multiplexor de dos entradas:

$$Y = \begin{cases} A & S = 0 \\ B & S = 1 \end{cases}$$

Implementación usando assign

```
module mux2 (  
    input wire A,  
    input wire B,  
    input wire S,  
    output wire Y  
);  
  
assign Y = S ? B : A;  
  
endmodule
```

La expresión $S ? B : A$ corresponde al operador ternario de Verilog.

Implementación usando always

```
module mux2_always (  
    input wire A,  
    input wire B,  
    input wire S,  
    output reg Y  
);  
  
always @(*) begin  
    if (S)  
        Y = B;  
    else  
        Y = A;  
    end  
  
endmodule
```

Ambos códigos sintetizan el mismo multiplexor.

4.6. Uso de case

La instrucción `case` permite describir circuitos combinacionales con múltiples condiciones.

Ejemplo: multiplexor 4 a 1

Un multiplexor de cuatro entradas puede representarse como:

$$Y = I_0 \overline{S_1} \overline{S_0} + I_1 \overline{S_1} S_0 + I_2 S_1 \overline{S_0} + I_3 S_1 S_0$$

Implementación en Verilog:

```
module mux4 (  
    input wire [1:0] S,  
    input wire [3:0] I,  
    output reg Y  
);  
  
always @(*) begin  
    case (S)  
        2'b00: Y = I[0];  
        2'b01: Y = I[1];  
        2'b10: Y = I[2];  
        2'b11: Y = I[3];  
    endcase  
end  
  
endmodule
```

4.7. Buenas prácticas en lógica combinacional

Al diseñar lógica combinacional en Verilog es recomendable seguir ciertas prácticas:

- Utilizar `always @(*)` para evitar errores en la lista de sensibilidad.
- Asignar todas las salidas en todos los caminos posibles.
- Evitar inferencia accidental de *latches*.
- Mantener claridad en el código HDL.

4.8. Resumen

La lógica combinacional describe circuitos cuyas salidas dependen únicamente de las entradas actuales. En Verilog estos circuitos pueden implementarse mediante:

- asignaciones continuas (`assign`)
- bloques combinacionales (`always @(*)`)

Comprender la relación entre las expresiones en HDL y los circuitos lógicos es fundamental para diseñar hardware correcto y eficiente.

4.9. Ejercicios

- Siete segmentos

- Descripción de un mux
- Descripción de una función booleana
- Operaciones sobre un bus
- Comparación de cantidades
- Concatenación y replicación

4.10. Laboratorio

Lab: <https://www.overleaf.com/read/tgmnhgxpzqs#0a83eb>

5.1. Introducción

Antes de implementar un diseño digital en una FPGA, es fundamental verificar que su comportamiento sea correcto. Para ello se utilizan herramientas de simulación que permiten observar cómo responde el circuito ante diferentes combinaciones de entrada.

Un **banco de pruebas** o *testbench* es un módulo escrito en HDL cuyo propósito es verificar el funcionamiento de otro módulo, conocido como *design under test* (DUT).

A diferencia de los módulos destinados a síntesis, los bancos de prueba no representan hardware real, sino que se utilizan únicamente durante la simulación. Su función principal es generar estímulos de entrada y observar las salidas del circuito.

El flujo típico de verificación se muestra a continuación:

Diseño HDL → Testbench → Simulación → Análisis de resultados

Durante la simulación, el testbench aplica diferentes combinaciones de entrada al diseño y permite verificar si las salidas corresponden al comportamiento esperado.

5.2. Estructura de un banco de pruebas

Un banco de pruebas suele incluir los siguientes elementos:

- Instanciación del módulo a verificar (DUT)
- Generación de estímulos de entrada

- Observación o registro de señales
- Control del tiempo de simulación

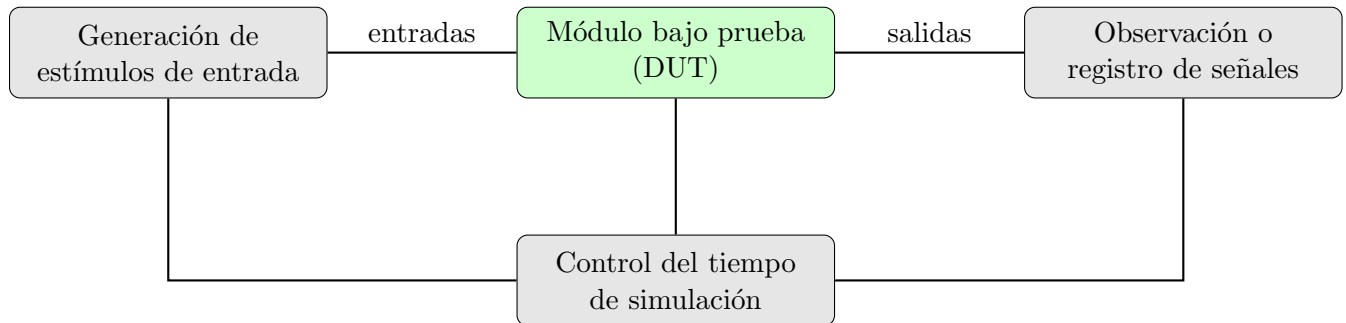


Figura 5.1: Estructura general de un banco de pruebas.

Una característica importante de los testbenches es que **no tienen puertos de entrada ni salida**, ya que no interactúan con otros módulos.

5.3. Ejemplo de diseño a verificar

Considere el siguiente módulo combinacional que implementa una compuerta XOR.

```

module xor_gate (
    input wire A,
    input wire B,
    output wire Y
);

assign Y = A ^ B;

endmodule
  
```

Este módulo será el circuito que deseamos verificar mediante simulación.

5.4. Creación de un banco de pruebas

Para probar el módulo anterior se puede escribir el siguiente testbench.

```

`timescale 1ns/1ps

module xor_gate_tb;

reg A;
reg B;
wire Y;
  
```

```
xor_gate dut (  
    .A(A),  
    .B(B),  
    .Y(Y)  
);  
  
initial begin  
    A = 0; B = 0; #10;  
    A = 0; B = 1; #10;  
    A = 1; B = 0; #10;  
    A = 1; B = 1; #10;  
  
    $finish;  
end  
  
endmodule
```

En este ejemplo se aplican las cuatro combinaciones posibles de las entradas A y B , permitiendo verificar que la salida corresponda a la función XOR.

5.5. Tipos de señales en un testbench

En los bancos de prueba se utilizan dos tipos principales de señales.

Señales de estímulo

Las entradas del módulo bajo prueba se declaran como **reg**, ya que sus valores serán modificados durante la simulación.

```
reg A;  
reg B;
```

Señales observadas

Las salidas del módulo se declaran como **wire**, ya que son conducidas por el DUT.

```
wire Y;
```

5.6. Control del tiempo en simulación

El operador **#** permite introducir retardos en la simulación.

```
#10;
```

Este retardo indica que deben transcurrir 10 unidades de tiempo antes de ejecutar la siguiente instrucción.

La directiva:

```
'timescale 1ns/1ps
```

define la escala de tiempo utilizada en la simulación. En este caso:

- unidad de tiempo: 1 nanosegundo
- precisión temporal: 1 picosegundo

5.7. Visualización de señales

Durante la simulación es posible imprimir información en la consola mediante las funciones del sistema.

`$display`

Permite imprimir valores de señales.

```
$display("A=%b B=%b Y=%b", A, B, Y);
```

`$monitor`

Imprime automáticamente los valores cuando alguna señal cambia.

```
$monitor("Tiempo=%0t A=%b B=%b Y=%b", $time, A, B, Y);
```

Estas funciones son útiles para depurar el comportamiento del circuito.

5.8. Generación automática de formas de onda

Para analizar las señales gráficamente, los simuladores permiten generar archivos de formas de onda.

Esto se realiza mediante las funciones:

```
$dumpfile("wave.vcd");  
$dumpvars(0, xor_gate_tb);
```

El archivo generado puede visualizarse posteriormente con herramientas como GTKWave.

5.9. Ejemplo completo de banco de pruebas

A continuación se muestra un testbench más completo que incluye visualización de señales y generación de formas de onda.

```
'timescale 1ns/1ps

module xor_gate_tb;

reg A;
reg B;
wire Y;

xor_gate dut (
    .A(A),
    .B(B),
    .Y(Y)
);

initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, xor_gate_tb);

    $monitor("t=%0t A=%b B=%b Y=%b", $time, A, B, Y);

    A = 0; B = 0; #10;
    A = 0; B = 1; #10;
    A = 1; B = 0; #10;
    A = 1; B = 1; #10;

    $finish;
end

endmodule
```

Este banco de pruebas permite verificar completamente el comportamiento del módulo XOR.

5.10. Buenas prácticas

Al diseñar bancos de prueba es recomendable seguir ciertas prácticas:

- Probar todas las combinaciones relevantes de entrada.
- Utilizar mensajes de depuración mediante `$display` o `$monitor`.
- Generar archivos de forma de onda para facilitar el análisis.
- Mantener los testbenches separados del código sintetizable.

5.11. Resumen

Los bancos de prueba permiten verificar el comportamiento de un diseño digital antes de su implementación en hardware.

Un testbench genera estímulos de entrada, observa las salidas del circuito y permite analizar su funcionamiento mediante simulación.

El uso sistemático de simulación es una práctica esencial en el diseño digital moderno, ya que permite detectar errores en etapas tempranas del desarrollo.

5.12. Ejercicios

- Testbench para una función booleana
- Testbench para comparacion de cantidades (ejercicio anterior)
- Testbench para un contador con prints
- Testbench con visualizador de señales

5.13. Laboratorio

Lab: <https://www.overleaf.com/read/cqfvkknymkgt#c8b390b>

6.1. Introducción

A diferencia de los circuitos combinacionales, en los circuitos secuenciales las salidas dependen no solo de las entradas actuales, sino también del historial previo del sistema. Esto se logra mediante elementos de memoria que permiten almacenar información entre ciclos de operación.

Formalmente, un circuito secuencial puede representarse como:

$$Y(t) = f(X(t), S(t))$$

$$S(t+1) = g(X(t), S(t))$$

donde:

- $X(t)$ representa las entradas del sistema
- $Y(t)$ representa las salidas
- $S(t)$ representa el estado interno del circuito

Los circuitos secuenciales se utilizan ampliamente en sistemas digitales para implementar:

- registros
- contadores
- máquinas de estados
- controladores

En FPGAs, estos circuitos se implementan principalmente mediante **flip-flops sincronizados por reloj**.

6.2. Señales de reloj

La mayoría de los sistemas digitales modernos utilizan un reloj global que sincroniza todas las operaciones del circuito. El reloj es una señal periódica que determina cuándo se actualizan los elementos de memoria.

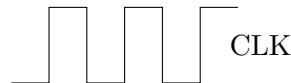


Figura 6.1: Señal de reloj digital.

Los flip-flops típicamente actualizan su estado en el **flanco de subida** del reloj.

6.3. Flip-Flop tipo D

El flip-flop tipo D es el elemento básico de almacenamiento utilizado en sistemas digitales sincronizados.

Su funcionamiento se describe mediante la siguiente relación:

$$Q(t + 1) = D(t)$$

Esto significa que el valor presente en la entrada D se almacena en la salida Q en el siguiente flanco del reloj.

Implementación en Verilog

El siguiente módulo describe un flip-flop tipo D.

```
module d_flipflop (  
    input wire clk,  
    input wire D,  
    output reg Q  
);  
  
always @(posedge clk)  
begin  
    Q <= D;  
end  
  
endmodule
```

En este ejemplo:

- `posedge clk` indica que el flip-flop se activa en el flanco de subida
- `<=` representa una asignación no bloqueante

6.4. Registro

Un registro es un conjunto de flip-flops que permiten almacenar múltiples bits. Por ejemplo, un registro de 4 bits puede representarse como:

$$Q[3:0] \leftarrow D[3:0]$$

Implementación en Verilog

```
module register4 (  
    input wire clk, input wire [3:0] D, output reg [3:0] Q  
);  
  
always @(posedge clk)  
begin  
    Q <= D;  
end  
  
endmodule
```

Cada bit del registro corresponde a un flip-flop independiente.

6.5. Registro con habilitación

En muchos sistemas es necesario controlar cuándo se actualiza un registro. Para ello se utiliza una señal de habilitación (*enable*).

Implementación en Verilog

```
module register_enable (  
    input wire clk, input wire en, input wire [3:0] D, output reg [3:0] Q  
);  
  
always @(posedge clk)  
begin  
    if (en)  
        Q <= D;  
end  
  
endmodule
```

Cuando la señal `en` es cero, el registro mantiene su valor previo.

6.6. Contadores

Un contador es un circuito secuencial que incrementa o decrementa su valor en cada ciclo de reloj.

Ejemplo: contador binario

```
module counter4 (  
    input wire clk,  
    output reg [3:0] count  
);  
  
always @(posedge clk)  
begin  
    count <= count + 1;  
end  
  
endmodule
```

Este módulo implementa un contador binario de 4 bits que incrementa su valor en cada flanco de subida del reloj.

6.7. Reset

Los circuitos secuenciales suelen incluir una señal de **reset** que permite inicializar el estado del sistema.

Reset síncrono

```
module counter_reset (  
    input wire clk,  
    input wire rst,  
    output reg [3:0] count  
);  
  
always @(posedge clk)  
begin  
    if (rst)  
        count <= 4'b0000;  
    else  
        count <= count + 1;  
end  
  
endmodule
```

Cuando la señal **rst** es activada, el contador se reinicia a cero.

6.8. Máquinas de estados finitos

Las máquinas de estados finitos (FSM) son circuitos secuenciales que cambian de estado en función de las entradas y del estado actual.

Formalmente, una FSM se define mediante:

- un conjunto finito de estados
- una función de transición
- una función de salida

Ejemplo de FSM

Considere una máquina de estados con dos estados que alterna entre ellos en cada ciclo de reloj.

```
// Declaracion de los estados y de los registros

localparam IDLE=2'b00, LOAD=2'b01, DONE=2'b10;
reg [1:0] state, next_state;

// Registro de estado:

always @(posedge clk or posedge rst) begin
    if (rst) state <= IDLE;
    else     state <= next_state;
end

// Logica de estado proximo:

always @(*) begin
    next_state = state; // default
    case (state)
        IDLE: next_state = LOAD;
        LOAD: next_state = DONE;
        DONE: next_state = IDLE;
        default: next_state = IDLE;
    endcase
end
```

6.9. Buenas prácticas

Al diseñar lógica secuencial en Verilog es recomendable seguir ciertas reglas:

- Utilizar `always @(posedge clk)` para describir flip-flops.
- Utilizar asignaciones no bloqueantes (`<=`) en lógica secuencial.

- Incluir señales de reset para inicializar los circuitos.
- Mantener separados los bloques de lógica combinacional y secuencial.

6.10. Resumen

La lógica secuencial permite diseñar circuitos cuyo comportamiento depende del historial previo del sistema. Estos circuitos utilizan elementos de memoria sincronizados por reloj, como flip-flops y registros.

En Verilog, los circuitos secuenciales se describen mediante bloques **always** activados por flancos de reloj. Estos bloques permiten implementar registros, contadores y máquinas de estados, que constituyen componentes fundamentales en el diseño de sistemas digitales complejos.

6.11. Ejercicios

- Creación de un contador
- Creación de un banco de registros síncronico
- Creación de registros universales
- Creación de una máquina de estados (máquina dispensadora)

6.12. Laboratorio

Lab: <https://www.overleaf.com/read/fcgcpwptpddn#be8bf2>

7.1. Introducción

Cuando un diseño digital se implementa en una FPGA, el circuito interactúa con el entorno externo mediante pines de entrada y salida. Estos pines permiten conectar dispositivos como botones, switches, LEDs, sensores y otros circuitos digitales. Sin embargo, las señales provenientes del exterior suelen presentar características que pueden causar problemas en el diseño digital, tales como:

- señales asíncronas respecto al reloj del sistema
- rebotes mecánicos en botones
- niveles lógicos incorrectos
- estados indeterminados en señales flotantes

Por esta razón, es necesario aplicar ciertas técnicas de diseño para garantizar un funcionamiento confiable del sistema.

7.2. Señales asíncronas

Una señal se considera **asíncrona** cuando no está sincronizada con el reloj del sistema digital. Ejemplos comunes incluyen:

- botones
- interruptores

- señales provenientes de otros dispositivos

Cuando una señal asíncrona se introduce directamente en lógica secuencial, puede provocar problemas de **metastabilidad**.

7.3. Metastabilidad

La metastabilidad ocurre cuando un flip-flop recibe una señal de entrada que cambia muy cerca del flanco del reloj. En estas condiciones, el flip-flop puede entrar en un estado intermedio temporal donde su salida no es ni lógica alta ni lógica baja.

Para reducir la probabilidad de metastabilidad se utilizan **sincronizadores**.

7.4. Sincronización de señales

La técnica más común para sincronizar una señal externa consiste en utilizar una cadena de flip-flops.

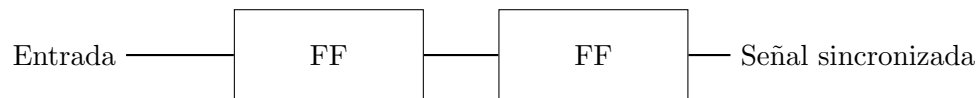


Figura 7.1: Sincronizador de dos etapas.

Este método reduce significativamente la probabilidad de propagación de estados metastables.

Implementación en Verilog

```
module synchronizer (  
    input wire clk,  
    input wire async_in,  
    output reg sync_out  
);  
  
reg sync_ff;  
  
always @(posedge clk)  
begin  
    sync_ff <= async_in;  
    sync_out <= sync_ff;  
end  
  
endmodule
```


7.5. Debouncing de botones

Los botones mecánicos presentan un fenómeno conocido como **rebote** (*bounce*). Cuando el botón es presionado, el contacto mecánico puede abrirse y cerrarse varias veces en un intervalo muy corto.

Esto genera múltiples transiciones lógicas en lugar de una sola.



Figura 7.2: Ejemplo de rebote en un botón.

Para eliminar estos rebotes se utilizan circuitos de **debouncing**.

7.6. Debouncer basado en contador

Una técnica común consiste en utilizar un contador que verifique que la señal permanezca estable durante un cierto número de ciclos de reloj.

```
module debouncer (  
    input wire clk,  
    input wire button,  
    output reg button_clean  
);  
  
reg [15:0] counter;  
  
always @(posedge clk)  
begin  
    if (button != button_clean)  
    begin  
        counter <= counter + 1;  
  
        if (counter == 16'hFFFF)  
        begin  
            button_clean <= button;  
            counter <= 0;  
        end  
    end  
    else  
        counter <= 0;  
end  
  
endmodule
```

Este circuito actualiza la señal solo si permanece estable durante varios ciclos de reloj.

7.7. Sincronización de señales de reset

Las señales de reset también pueden provenir del exterior del sistema y, por lo tanto, ser asíncronas.

Un enfoque común consiste en utilizar:

- reset asíncrono
- liberación sincronizada

Ejemplo de sincronización de reset

```
module reset_synchronizer (  
    input wire clk,  
    input wire async_rst,  
    output reg sync_rst  
);  
  
reg sync_ff;  
  
always @(posedge clk)  
begin  
    sync_ff <= async_rst;  
    async_rst <= sync_ff;  
end  
  
endmodule
```

Este circuito garantiza que la señal de reset se libere de manera sincronizada con el reloj.

7.8. Constraints de I/O

Las herramientas de diseño para FPGA utilizan archivos de restricciones para definir cómo se conectan las señales del diseño con los pines físicos del dispositivo.

En Vivado, estas restricciones se definen mediante archivos **XDC**.

Asignación de pines

Un ejemplo de asignación de pin es:

```
set_property PACKAGE_PIN W5 [get_ports clk]  
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

Esto asigna la señal `clk` al pin físico `W5` utilizando el estándar lógico `LVCMOS33`.

7.9. Niveles lógicos

Los niveles lógicos definen el rango de voltajes utilizados para representar valores binarios. Algunos estándares comunes incluyen:

- LVCMOS33
- LVCMOS18
- LVTTL

La elección del estándar depende del hardware externo conectado a la FPGA.

7.10. Resistencias pull-up y pull-down

Cuando una señal de entrada no está conectada activamente a un nivel lógico, puede quedar en un estado indeterminado. Para evitar esto se utilizan resistencias de polarización:

- **pull-up:** fuerza la señal a nivel lógico alto
- **pull-down:** fuerza la señal a nivel lógico bajo

En Vivado estas resistencias pueden configurarse mediante constraints.

Ejemplo de pull-up

```
set_property PULLUP true [get_ports button]
```

Ejemplo de pull-down

```
set_property PULLDOWN true [get_ports switch]
```

7.11. Resumen

Las interfaces de entrada y salida requieren consideraciones especiales para garantizar el funcionamiento correcto del sistema digital.

Las señales externas deben sincronizarse con el reloj del sistema para evitar problemas de metastabilidad. Además, los botones mecánicos requieren circuitos de debouncing para eliminar rebotes.

Finalmente, los archivos de constraints permiten definir cómo se conectan las señales del diseño con los pines físicos de la FPGA, incluyendo configuraciones como niveles lógicos y resistencias de polarización.

7.12. Ejercicios

- Cómo se logra estabilizar una señal de un relay que conmuta a 1 Hz.
- Cómo se convierte un reset asincrónico a un reset sincrónico
- Crear un generador de pulso

7.13. Laboratorio

Lab: <https://www.overleaf.com/read/cqfvkknymkgt#c8b390>

8.1. Introducción

La microarquitectura de un sistema digital describe la organización interna del hardware que implementa un conjunto de operaciones o instrucciones. A diferencia de la arquitectura del sistema, que define el comportamiento visible para el programador, la microarquitectura se enfoca en cómo se implementan físicamente estas operaciones en el hardware.

En sistemas digitales implementados en FPGA o ASIC, el diseño de la microarquitectura influye directamente en aspectos como:

- Latencia
- Throughput
- Frecuencia máxima de operación
- Consumo de área

Existen tres microarquitecturas fundamentales ampliamente utilizadas en sistemas digitales:

- Arquitectura unicycle
- Arquitectura multiciclo
- Arquitectura segmentada (pipeline)

Cada una representa un compromiso distinto entre rendimiento, complejidad y uso de recursos.

8.2. Métricas de desempeño

Antes de analizar las distintas microarquitecturas es importante definir algunas métricas utilizadas para evaluar el desempeño de un sistema digital.

Throughput

Representa la cantidad de datos que un sistema puede procesar por unidad de tiempo o por ciclo de reloj.

$$Throughput = \frac{\text{Datos procesados}}{\text{Ciclo de reloj}}$$

Generalmente se expresa en bits por ciclo o en operaciones por segundo.

Latencia

La latencia corresponde al tiempo necesario para que una entrada produzca un resultado en la salida del sistema.

$$Latencia = \text{Número de ciclos entre entrada y salida}$$

Timing

El *timing* de un circuito está determinado por el retraso total entre dos flip-flops consecutivos. Este retraso incluye:

- retraso de la lógica combinacional
- clk-to-out
- skew del reloj
- retraso de ruteo
- tiempo de setup

Reducir la ruta crítica permite aumentar la frecuencia máxima del sistema.

8.3. Microarquitectura uniciclo

En una arquitectura uniciclo, cada operación o instrucción se ejecuta completamente dentro de un único ciclo de reloj.

Esto implica que todas las operaciones necesarias para completar la instrucción deben realizarse dentro de la misma ruta combinacional.

Características

- baja latencia
- frecuencia de operación baja
- ruta crítica extensa
- consumo de área moderado

El periodo del reloj debe ser lo suficientemente largo para permitir que todas las operaciones se completen dentro del mismo ciclo.

Ejemplo conceptual

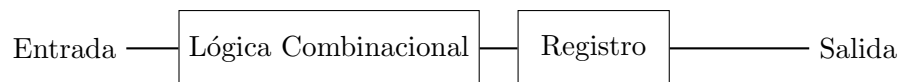


Figura 8.1: Arquitectura uniciclo simplificada

En este caso, toda la operación ocurre dentro de un solo ciclo de reloj.

8.4. Microarquitectura multiciclo

La arquitectura multiciclo divide la ejecución de una operación en múltiples ciclos de reloj. En cada ciclo se ejecuta una parte del proceso.

Este enfoque permite reutilizar recursos de hardware como la ALU o la memoria.

Características

- menor consumo de área
- reutilización de recursos
- mayor latencia
- frecuencia de operación más alta

El control del sistema se vuelve más complejo porque es necesario coordinar las distintas etapas mediante una máquina de estados.

Ejemplo conceptual

Cada etapa se ejecuta en un ciclo de reloj diferente.



Figura 8.2: Arquitectura multiciclo

8.5. Microarquitectura segmentada

La arquitectura segmentada, o pipeline, divide el procesamiento en varias etapas que pueden ejecutarse simultáneamente sobre diferentes datos.

Esto permite incrementar significativamente el throughput del sistema.

Características

- alta frecuencia de operación
- alto throughput
- mayor consumo de área
- mayor complejidad de control

En una arquitectura pipeline, cada etapa procesa una parte distinta de la operación.

Ejemplo conceptual

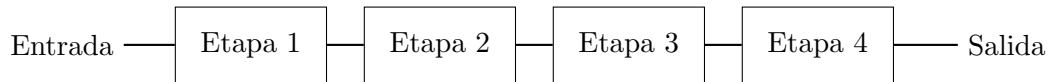


Figura 8.3: Arquitectura pipeline

En cada ciclo de reloj, una nueva entrada puede ingresar al pipeline mientras las anteriores avanzan por las distintas etapas.

8.6. Comparación de microarquitecturas

Característica	Uniciclo	Multiciclo	Pipeline
Latencia	Baja	Alta	Media
Frecuencia	Baja	Alta	Alta
Throughput	Bajo	Medio	Alto
Área	Media	Baja	Alta
Complejidad	Baja	Media	Alta

8.7. Compromisos de diseño

El diseño de microarquitecturas implica elegir el compromiso adecuado entre:

- latencia
- frecuencia de operación
- consumo de recursos
- complejidad del sistema

Por ejemplo:

- Si el área es una restricción importante, se suele utilizar arquitectura multiciclo.
- Si el throughput es la prioridad, se utilizan arquitecturas segmentadas.
- Si la latencia debe ser mínima, puede preferirse una arquitectura uniciclo.

En aplicaciones de cómputo de alto desempeño es común utilizar pipelines y paralelismo para maximizar el uso del hardware disponible.

8.8. Resumen

Las microarquitecturas determinan la forma en que un sistema digital organiza su hardware para ejecutar operaciones.

Las arquitecturas uniciclo, multiciclo y segmentadas representan diferentes estrategias para optimizar el rendimiento de un sistema digital.

La elección de la microarquitectura depende de los objetivos del diseño, incluyendo restricciones de área, latencia, frecuencia de operación y consumo de energía.

Completar con el siguiente material:<https://docs.google.com/document/d/1MoSSkA0QEgTv0mUIGbLof0Cmnd/edit?usp=sharing>

Poner las definiciones y verificar si lo que está aquí es válido. Completar aquello que no aplique. La idea es completar con ejemplos de ese material.

Para renderizar los circuitos puede usar CircuiTikz o usar TerosHDL para los esquematicos usando Yosys

8.9. Laboratorio

Lab: <https://www.overleaf.com/read/wgjhbvpwvdh#0ca72a>

9.1. Introducción

A medida que los sistemas digitales aumentan en complejidad, resulta impráctico diseñarlos como un único bloque monolítico. En su lugar, se adopta un enfoque estructurado basado en la división del sistema en múltiples niveles jerárquicos.

El **diseño digital multinivel** consiste en organizar un sistema en módulos interconectados, donde cada módulo cumple una función específica y puede ser diseñado, verificado y reutilizado de manera independiente, sin necesidad de tomar en cuenta el sistema entero.

Este enfoque es fundamental en el desarrollo de sistemas en FPGA y ASIC, ya que permite:

- manejar la complejidad del diseño para mantenerlo encapsulado
- mejorar la reutilización de componentes en otros sistemas o actualizaciones del mismo
- facilitar la verificación de un módulo en aislamiento
- acelerar el desarrollo al permitir que los miembros de un equipo trabajen en paralelo en módulos distintos

9.2. Concepto de jerarquía

La jerarquía en diseño digital implica estructurar el sistema en distintos niveles de abstracción.

- Nivel superior (*top-level*)
- Módulos intermedios

- Módulos básicos

Ejemplo conceptual

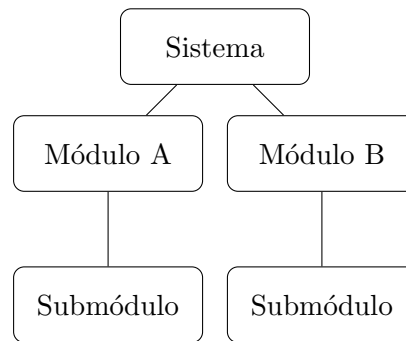


Figura 9.1: Estructura jerárquica de un sistema digital

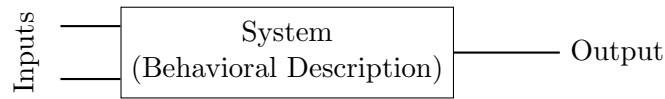
El diseño jerárquico es esencial por varias razones:

- **Escalabilidad:** Permite diseñar sistemas grandes dividiéndolos en bloques más pequeños y manejables.
- **Reutilización:** Los módulos pueden reutilizarse en diferentes proyectos o partes del mismo sistema.
- **Mantenibilidad:** Los cambios en un módulo no afectan directamente a otros, siempre que la interfaz se mantenga.
- **Verificación modular:** Cada módulo puede verificarse de forma independiente mediante testbenches.

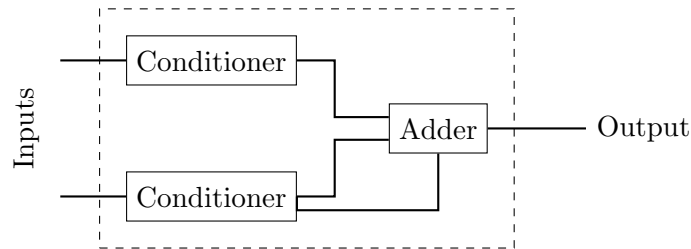
9.3. Principios de diseño

El proceso de diseño generalmente se fundamenta en la metodología de ingeniería para la resolución de problemas. Generalmente, el top-down sigue la siguiente secuencia:

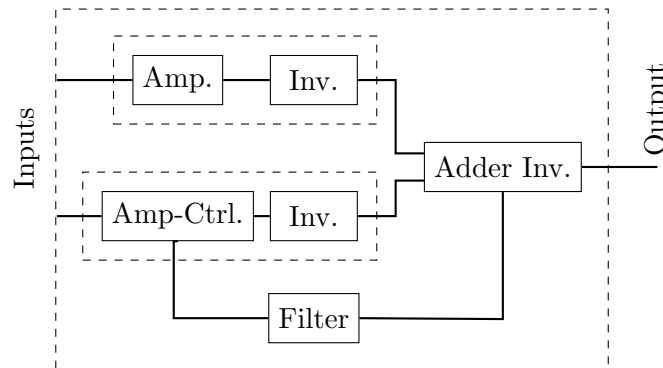
1. **Entender el problema:** se requiere un levantamiento de requisitos del sistema, que definen las entradas, salidas, control y el comportamiento del sistema. Asimismo, define aspectos físicos y eléctricos del sistema en cuestión, como la tecnología, energía de operación, entre otros.
2. **Levantamiento de las interfaces:** consiste en enumerar las entradas, las salidas y el control del sistema, como si se tratase de una caja negra. Podemos llamar a este diseño como el diagrama de nivel 1, donde todo se ve como si fuera una caja negra.



3. **Definición del funcionamiento interno:** consiste en abrir la caja negra y comenzar a incorporar una arquitectura en el sistema, que se puede ilustrar como un diagrama de bloques funcionales. En este punto, componemos la arquitectura como una ruta de datos (datapath), que realiza transformaciones de los datos de una forma casi funcional. Tomaremos de ejemplo que S3 corresponde a un sumador de señal.



4. **Definición de la implementación:** cuando la funcionalidad está descrita mediante una arquitectura, el siguiente paso es la microarquitectura, que define cómo se implementará cada bloque para brindar la funcionalidad requerida. Un ejemplo de ello es que existen distintas implementaciones para un sumador, como el sumador inversor o no inversor. También, es permitido dividir los bloques funcionales en bloques más pequeños.



9.4. Flujo de diseño multinivel

Por otra parte, el desarrollo de un sistema jerárquico sigue típicamente las siguientes etapas (de acuerdo con la Figura 9.2):

1. Definición de la arquitectura del sistema
2. División en módulos

3. Definición de interfaces
4. Implementación de módulos
5. Verificación individual
6. Integración del sistema
7. Verificación global

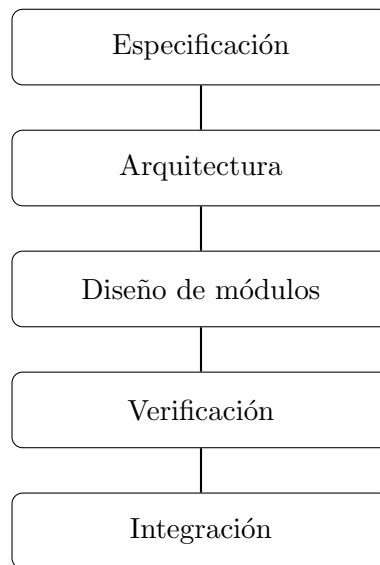


Figura 9.2: Flujo de diseño multinivel

1. Definición de la arquitectura del sistema

En esta etapa se define el comportamiento global del sistema y cómo se organizarán sus componentes principales.

Se debe responder a preguntas como:

- ¿Qué hace el sistema?
- ¿Qué entradas y salidas tiene?
- ¿Qué tipo de procesamiento realiza?
- ¿Cuál es la frecuencia de operación?

También se identifican los bloques principales del sistema, por ejemplo:

- unidad de control
- datapath
- memoria
- interfaces externas

Resultado: un diagrama de bloques de alto nivel.

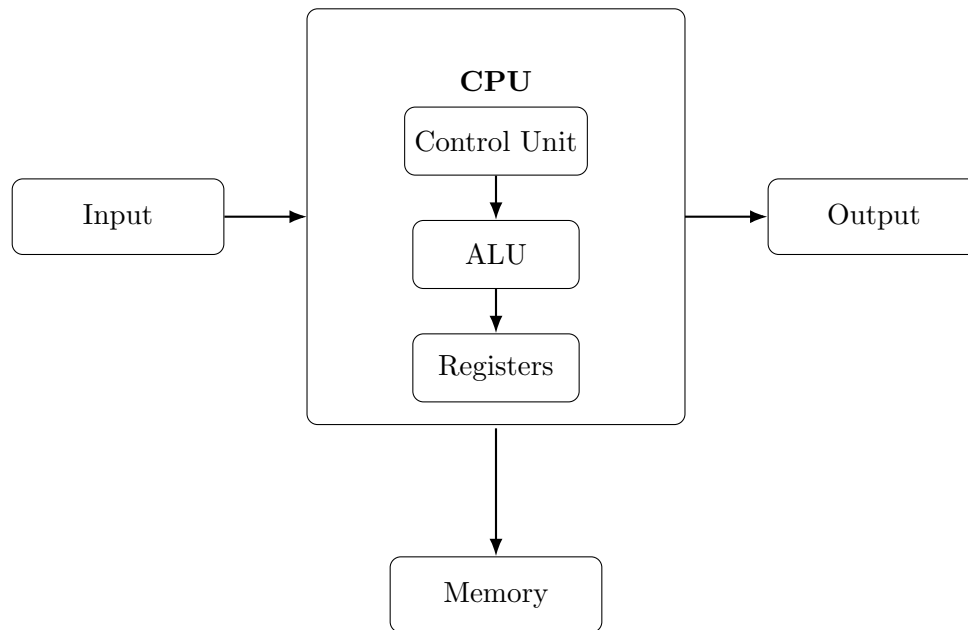


Figura 9.3: Diagrama de bloques simplificado de una computadora con componentes internos de la CPU.

2. División en módulos

Una vez definida la arquitectura, el sistema se divide en módulos más pequeños. Cada módulo debe tener una función clara, ser independiente de otros módulos y reutilizables. Algunos ejemplos son:

- ALU
- registros
- unidad de control
- memoria

Principio clave: dividir el sistema hasta que cada módulo sea fácil de entender e implementar. Generalmente, se dividen en bloques genéricos comunes del diseño digital.

3. Definición de interfaces

Las interfaces definen cómo se comunican los módulos entre sí. Se deben especificar:

- señales de entrada y salida
- ancho de datos
- señales de control
- protocolo de comunicación (si aplica)

Ejemplo:

```
input wire clk,  
input wire [7:0] data_in,  
output wire valid
```

Importante: una interfaz bien definida permite modificar el interior de un módulo sin afectar al resto del sistema. Asimismo, hay bloques que tienen interfaces estándar, como AXI o serie. Es recomendado empaquetarlos en el caso de SystemVerilog.

4. Implementación de módulos

En esta etapa se escribe el código HDL de cada módulo. Para esta tarea se recomienda comenzar por los módulos más simples, reutilizar componentes existentes y mantener código claro y documentado (ir documentando conforme se implementan los módulos y puede ayudarse con la IA). Por ejemplo:

```
/**  
 * @brief Multiplexor 2 a 1  
 *  
 * Este mdulo implementa un multiplexor de dos entradas.  
 * Selecciona entre las seales A y B dependiendo del valor de la seal S.  
 *  
 * Si S = 0 la salida es A  
 * Si S = 1 la salida es B  
 */  
module mux2 (  
    /** @brief Entrada A */  
    input wire A,  
    /** @brief Entrada B */  
    input wire B,  
    /** @brief Senal de seleccion */  
    input wire S,  
  
    /** @brief Salida del multiplexor */
```



```

    output wire Y
);

    /** @brief Lógica combinacional del multiplexor */
    assign Y = S ? B : A;

endmodule

```

Este estilo de comentarios tiene una sintaxis, lo que permite la auto-generación de la documentación, que se convierte en LaTeX o sitios web. Puede encontrar más detalles en Documenter.

5. Verificación individual

Cada módulo debe verificarse de forma independiente mediante un testbench. Esto permite:

- detectar errores temprano
- simplificar la depuración
- validar el comportamiento esperado

La estrategia usualmente consiste en probar casos normales, casos extremos y verificar las condiciones de error que no están permitidas en el sistema o módulo.

6. Integración del sistema

Una vez verificados los módulos, se conectan en un módulo superior. Ejemplo:

```

module top (
    input wire clk,
    input wire [3:0] A,
    input wire [3:0] B,
    output wire [3:0] Y
);

    wire [3:0] sum;

    adder u1 (.A(A), .B(B), .Y(sum));
    register u2 (.clk(clk), .D(sum), .Q(Y));

endmodule

```

En esta etapa pueden aparecer errores relacionados con:

- conexiones incorrectas
- sincronización
- incompatibilidad de señales

7. Verificación global

Finalmente, se verifica el sistema completo mediante un testbench a nivel top. Esto incluye la validación funcional completa, la verificación de interacción entre módulos y la evaluación de rendimiento (latencia, throughput). Es importante destacar que, aunque los módulos funcionen individualmente, el sistema completo puede presentar errores.

9.5. Buenas prácticas de diseño

Es importante seguir buenas prácticas desde el inicio del proyecto, ya que nos permiten mantener el orden, la calidad y la funcionalidad. Al final, seguir una estrategia estructurada permite ir lo suficientemente rápido con pasos firmes, sin la necesidad de devolverse para reiteraciones. Algunas buenas prácticas incluyen:

- **Modularidad:** Cada módulo debe tener una única responsabilidad.
- **Encapsulamiento:** Evitar exponer detalles internos del módulo.
- **Nombres descriptivos:** Utilizar nombres claros para módulos y señales.
- **Separación de lógica:** Separar lógica combinacional y secuencial.
- **Uso de parámetros:** Permitir reutilización mediante parametrización.

```
module adder #(parameter N = 4) (  
    input wire [N-1:0] A,  
    input wire [N-1:0] B,  
    output wire [N-1:0] Y  
);  
  
assign Y = A + B;  
  
endmodule
```

10.1. Introducción

En el diseño de sistemas digitales modernos, especialmente en entornos de FPGA y ASIC, es común trabajar con múltiples archivos, herramientas y flujos de procesamiento. La ejecución manual de estas tareas puede ser propensa a errores y poco eficiente.

El **scripting** permite automatizar tareas repetitivas mediante el uso de lenguajes como:

- Bash
- Python
- Tcl
- Awk

Estas herramientas permiten crear, modificar, analizar y transformar datos de manera eficiente, facilitando el desarrollo de sistemas complejos.

¿Qué es el scripting?

El scripting es la práctica de escribir programas pequeños (scripts) que automatizan tareas que normalmente se realizarían de forma manual.

Un **script** es una secuencia de instrucciones que:

- se ejecutan en orden
- interactúan con el sistema operativo o herramientas

- manipulan archivos o datos

A diferencia de programas tradicionales, los scripts suelen enfocarse en automatización, integración de herramientas y procesamiento de datos.

Motivación

Considere el siguiente escenario:

1. Tiene 50 archivos Verilog.
2. Debe compilarlos.
3. Ejecutar simulación.
4. Revisar errores en los logs.
5. Generar un reporte.

Hacer esto manualmente implica repetir comandos, cometer errores humanos y perder tiempo. Con scripting, este proceso puede automatizarse completamente: un solo comando ejecuta todo el flujo, compilando cada uno de los archivos en Verilog, ejecutando la simulación, recolectando los resultados y generando un reporte de aquellas implementaciones que funcionan o no.

Scripting como composición de herramientas

El scripting no consiste en hacer todo desde cero, sino en **componer herramientas existentes**. Por ejemplo:

- Bash ejecuta comandos
- Awk procesa texto
- Python analiza datos
- Tcl controla herramientas EDA

Un script conecta estas piezas.

Modelo mental del scripting

Una forma útil de entender scripting es verlo como una **tubería de datos**:

Entrada → Procesamiento → Salida

Por ejemplo, consideremos que tenemos una serie de simulaciones y solo queremos ver cuáles configuraciones muestran error. Los resultados de las simulaciones fueron guardados en un archivo log.txt con la siguiente información:

```
RESULTADO CONFIGURATION
PASS      AND-22nm-APPROX
ERROR     AND-22nm-EXACT
PASS      OR-22nm-APPROX
ERROR     XOR-22nm-EXACT
ERROR     OR-22nm-EXACT
```

Para este filtrado, procedemos a utilizar un comando que abre el log (cat), luego pasa sus resultados a un procesador (grep) y el resultado luego se pasa a un filtro (awk):

```
cat log.txt | grep ERROR | awk '{print $2}'
```

Esto produce el siguiente resultado:

```
AND-22nm-EXACT
XOR-22nm-EXACT
OR-22nm-EXACT
```

Basado en este concepto, podemos dirigirnos hacia la automatización, que consiste en ejecutar una secuencia de acciones sin intervención manual. En el contexto de diseño digital, esto incluye:

- Compilar proyectos
- Ejecutar simulaciones
- Procesar resultados
- Generar reportes
- Manipular archivos de diseño

10.2. Scripting con Bash

Bash es un intérprete de comandos ampliamente utilizado en sistemas Unix/Linux. Utiliza la misma sintaxis de los comandos que se escriben en la terminal, como `cp`, `ls`, `cd`, `mv`, `cat`, entre otros.

Navegación en el sistema de archivos

```
pwd      # Muestra el directorio actual
ls       # Lista archivos y directorios
ls -l    # Lista detallada
ls -a    # Incluye archivos ocultos
cd dir   # Cambia de directorio
cd ..    # Ir al directorio padre
cd ~     # Ir al directorio home
```

Manipulación de archivos

```
touch file.txt      # Crear archivo vacio
cp file.txt copy.txt # Copiar archivo
mv file.txt new.txt  # Renombrar o mover archivo
rm file.txt          # Eliminar archivo
cat file.txt         # Mostrar contenido
less file.txt        # Ver contenido paginado
```

Manipulación de directorios

```
mkdir dir           # Crear directorio
rmdir dir           # Eliminar directorio vacio
rm -r dir           # Eliminar directorio con contenido
```

Búsqueda de archivos

```
find . -name "*.v"  # Buscar archivos con extension .v
find . -type f      # Buscar archivos
find . -type d      # Buscar directorios
```

Redirección y tuberías

```
command > file.txt    # Redirigir salida
command >> file.txt    # Aadir salida
command < file.txt    # Entrada desde archivo

command1 | command2   # Pipe (tubera)
```

Ejemplo:

```
cat log.txt | grep ERROR
```

Búsqueda de texto

```
grep "ERROR" log.txt
grep -i "error" log.txt  # Ignorar mayusculas
grep -r "main" .         # Bsqueda recursiva
```

Variables

```
VAR=10
echo $VAR
```

Condicionales

```
if [ $VAR -eq 10 ]; then
    echo "Es 10"
else
    echo "No es 10"
fi
```

Ciclos

```
for file in *.v
do
    echo "Archivo: $file"
done
```

Ejecución de scripts

Para ejecutar un script, el archivo que contiene el script debe marcarse como ejecutable, añadiendo los permisos correspondientes y luego se puede ejecutar:

```
chmod +x script.sh
./script.sh
```

Código de retorno

Cada comando devuelve un código de estado:

- 0: éxito
- distinto de 0: error

Ejemplo:

```
if [ $? -ne 0 ]; then
    echo "Error"
fi
```

Comandos útiles adicionales

<code>echo "Hola mundo"</code>	<i># Imprimir texto</i>
<code>clear</code>	<i># Limpiar pantalla</i>
<code>history</code>	<i># Historial de comandos</i>
<code>whoami</code>	<i># Usuario actual</i>
<code>date</code>	<i># Fecha y hora</i>

10.3. Scripting con TCL

Tcl es ampliamente utilizado en herramientas EDA como Vivado, Quartus, Synopsis, entre otros. Para los diseñadores digitales, conocer la existencia y papel de TCL es importante para automatizar procesos y generación de reportes.

Ejemplo básico

Para enlistar los archivos terminados en `.v` de un directorio e imprimir sus nombres, se puede utilizar:

```
set files [glob *.v]

foreach f $files {
    puts "Archivo: $f"
}
```

Para ponerlo en práctica, puede instalar el intérprete de TCL usando:

```
sudo apt install tcl
```

Luego, puede ejecutar el script usando:

```
tclsh script.tcl
```

Otra alternativa es crear un archivo usando el shebang, que permite indicar en la primera línea, cuál es el intérprete que se debe usar:

```
#!/usr/bin/env tclsh

set files [glob *.v]

foreach f $files {
    puts "Archivo: $f"
}
```

```
chmod +x script.tcl
./script.tcl
```

El shebang aplica para la mayoría de lenguajes interpretados y de scripting, como Python, Bash, TCL, entre otros.

Ejemplo en Vivado

Vivado (al igual que otras herramientas EDA) pone a disposición una serie de comandos personalizados dentro de TCL, definidos a través de funciones.

```
set_part "xc7a100tcsg324-1"
read_verilog adder.v
read_verilog calculator.v
```



```
synth_design -top calculator
report_utilization
exit
```

Para ejecutarlo, se puede usar Vivado en modo TCL:

```
# El archivo se llama synth.tcl
vivado -mode tcl -script synth.tcl
```

Eso va a generar un archivo .log con los resultados.

Sintaxis básica

La estructura general de un comando en TCL es:

```
comando argumento1 argumento2 ...
```

Ejemplo:

```
puts "Hola mundo"
```

Variables

Las variables se asignan con `set`:

```
set x 10
puts $x
```

Listas

Las listas son estructuras fundamentales en TCL.

```
set lista {a b c d}
puts [lindex $lista 0] # Acceso al primer elemento
```

Operaciones comunes:

```
llength $lista # Tamano de la lista
lappend lista e # Agregar elemento
```

Condicionales

```
if {$x == 10} {
    puts "Es 10"
} else {
    puts "No es 10"
}
```

Ciclos

Existen distintos tipos de ciclos, como el `for`, que funciona como el ciclo en C y el `foreach`, que itera sobre elementos de una lista.

```
set files {a.v b.v c.v}

foreach f $files {
    puts "Archivo: $f"
}
```

```
for {set i 0} {$i < 5} {incr i} {
    puts "i = $i"
}
```

Operaciones aritméticas

Se realizan con el comando `expr`:

```
set x 5
set y 3
set z [expr $x + $y]
puts $z
```

Manipulación de archivos

Para abrir archivos:

```
set f [open "archivo.txt" r]
set contenido [read $f]
close $f
```

Para escribir archivos:

```
set f [open "salida.txt" w]
puts $f "Hola mundo"
close $f
```

Para buscar archivos:

```
set files [glob *.v]

foreach f $files {
    puts $f
}
```

Procedimientos

TCL permite definir funciones llamadas procedimientos, que son bloques de instrucciones y son útiles para tareas repetitivas.

```
proc saludo {nombre} {  
    puts "Hola $nombre"  
}  
  
saludo "Juan"
```

10.4. Procesamiento de datos con Awk

Awk es una herramienta de línea de comandos diseñada para procesar y analizar archivos de texto estructurado. Es especialmente útil cuando se trabaja con archivos de logs, reportes o datos tabulares. A diferencia de otros lenguajes, Awk está orientado a trabajar línea por línea, permitiendo filtrar, transformar y extraer información de forma eficiente.

Modelo de ejecución

Awk procesa los archivos siguiendo este esquema:

1. Lee una línea del archivo
2. Divide la línea en campos
3. Ejecuta una acción si se cumple una condición

La estructura general es:

```
awk 'condicion {accion}' archivo.txt
```

Campos y variables

Por defecto, Awk separa los campos por espacios o tabulaciones.

- \$0: línea completa
- \$1: primer campo
- \$2: segundo campo
- \$NF: último campo
- NR: número de línea

Ejemplo:

```
awk '{print $1}' archivo.txt
```

Imprime la primera columna del archivo. Ahora, considere el siguiente archivo:

```
10 OK
20 OK
30 ERROR
40 OK
```

Para imprimir solo las líneas que contienen ERROR:

```
awk '/ERROR/ {print $0}' archivo.txt
```

Condiciones

Awk permite usar condiciones numéricas:

```
awk '$1 > 20 {print $0}' archivo.txt
```

Esto imprime las líneas donde el primer campo es mayor que 20.

Operaciones aritméticas

```
awk '{print $1 * 2}' archivo.txt
```

Contadores

Ejemplo: contar líneas con ERROR:

```
awk '/ERROR/ {count++} END {print count}' archivo.txt
```

Bloques BEGIN y END

- BEGIN: se ejecuta antes de procesar el archivo
- END: se ejecuta después

Ejemplo:

```
awk 'BEGIN {print "Inicio"} {print $0} END {print "Fin"}' archivo.txt
```

Formato de salida

```
awk '{printf "Valor: %d\n", $1}' archivo.txt
```

Cambio de separador

Para archivos CSV:

```
awk -F',' '{print $1}' datos.csv
```

10.5. Flujo automatizado

Un flujo típico puede combinar múltiples herramientas:

```
#!/bin/bash

# Ejecutar simulacion
vivado -mode tcl -script synth.tcl

# Procesar resultados
grep -i "Slice LUTs" vivado.log | awk '{print $13}' >> luts.txt

# Generar reporte
echo "Simulacion completada. Ver archivo luts.txt"
```

10.6. Buenas prácticas

- Usar scripts para tareas repetitivas
- Mantener scripts simples y modulares
- Documentar los scripts
- Validar entradas y salidas

10.7. Resumen

El scripting es una herramienta fundamental para automatizar tareas en el diseño digital. Permite mejorar la productividad, reducir errores y facilitar el manejo de grandes volúmenes de datos.

El uso combinado de Bash, Python, Tcl y Awk proporciona un entorno poderoso para la automatización de flujos de trabajo en ingeniería electrónica.

Control de Versiones con Git

En el desarrollo de sistemas digitales, software, scripts y documentación técnica, los archivos cambian constantemente. Se corrigen errores, se agregan nuevas funcionalidades, se reorganiza el código y, en muchos casos, varias personas trabajan sobre el mismo proyecto. Sin un mecanismo de control, estos cambios pueden provocar problemas como:

- pérdida de versiones anteriores
- sobrescritura accidental de archivos
- dificultad para rastrear errores
- imposibilidad de saber quién realizó un cambio
- integración conflictiva entre el trabajo de varias personas

Para resolver estos problemas se utilizan sistemas de **control de versiones**.

Un sistema de control de versiones permite registrar la evolución de un conjunto de archivos a lo largo del tiempo, facilitando la recuperación de estados previos, la comparación entre versiones y la colaboración ordenada entre múltiples desarrolladores.

11.1. Git

Git es un sistema de control de versiones distribuido ampliamente utilizado en proyectos de software, hardware digital, documentación y automatización. Permite registrar cambios en archivos, crear historial del proyecto, trabajar en ramas independientes, colaborar con otras personas y fusionar cambios de manera controlada, como una evolución de un proyecto en equipo y de una

aplicación que se va extendiendo y mejorando en el tiempo.

A diferencia de otros sistemas, Git es **distribuido**, lo que significa que cada usuario posee una copia completa del historial del repositorio en su propia máquina.

11.2. ¿Por qué usar Git en diseño digital?

Aunque Git es muy conocido en desarrollo de software, también es extremadamente útil en ingeniería electrónica y diseño digital. Algunos ejemplos de archivos que pueden gestionarse con Git son:

- código Verilog o VHDL
- testbenches
- scripts
- documentación técnica
- reportes y configuraciones

El uso de Git en proyectos de FPGA y ASIC permite rastrear los cambios en módulos HDL, mantener scripts de simulación y síntesis, documentar implícitamente el progreso del proyecto al tener una historia del proyecto, coordinar trabajo en equipo y asociar resultados experimentales a versiones concretas del diseño.

11.3. Modelo mental básico de Git

Para comenzar a usar Git correctamente es importante entender sus conceptos fundamentales.

Repositorio

Un **repositorio** es el contenedor del proyecto y de su historial de cambios. En Git, el repositorio almacena:

- los archivos actuales
- el historial de versiones
- las ramas
- la información de seguimiento del proyecto

Commit

Un **commit** es una instantánea del proyecto en un momento específico. Cada commit tiene un identificador único que lo representa y comprime un conjunto de cambios, guardando el autor, fecha y mensaje descriptivo.

Branch

Una **rama** es una línea independiente de desarrollo. Estas permiten experimentar sin afectar la versión principal, desarrollar funcionalidades o módulos aislados y corregir errores de forma ordenada.

Merge

Un **merge** es la operación que integra los cambios de una rama en otra.

Remote

Un **repositorio remoto** es una copia del repositorio alojada en un servidor, por ejemplo en GitHub, GitLab o Bitbucket.

Working Directory

Es el directorio de trabajo actual, donde se editan los archivos.

Staging Area

Es una zona intermedia donde se preparan los cambios que serán incluidos en el siguiente commit.

Flujo básico

El flujo típico en Git es:

Archivo modificado → Staging Area → Commit

11.4. Primer flujo de trabajo con Git

A continuación se muestra un flujo mínimo de trabajo para comenzar a usar Git.

1. Inicializar un repositorio

```
git init
```

Este comando crea un repositorio Git en el directorio actual.

2. Ver el estado del repositorio

```
git status
```

Este es uno de los comandos más importantes para principiantes, ya que muestra:

- qué archivos han cambiado

- qué archivos están en staging
- qué archivos no están siendo seguidos por Git

3. Agregar archivos al staging area

```
git add archivo.v
```

Este comando admite *wildcards*, que son expresiones que permiten agregar multiples archivos que siguen un patrón en su nombre. Por ejemplo, *.v, que indica que se deben agregar los archivos que terminen en .v.

4. Crear un commit

```
git commit -m "Agrega multiplexor de 2 entradas"
```

5. Ver historial

Permite ver el historial de la rama del repositorio, destacando todos los commits realizados sobre la misma.

```
git log --graph
```

Versión resumida:

```
git log --oneline
```

Supóngase que se desea controlar versiones de un proyecto sencillo con archivos Verilog.

```
mkdir proyecto_fpga
cd proyecto_fpga
git init
touch mux2.v
git status
git add mux2.v
git commit -m "Agrega modulo mux2"
```

Luego, si se modifica el archivo:

```
git status
git diff
git add mux2.v
git commit -m "Corrige descripcion del multiplexor"
```

11.5. Comandos fundamentales

`git init`

Inicializa un nuevo repositorio.

`git clone`

Copia un repositorio remoto a la máquina local.

```
git clone https://github.com/usuario/proyecto.git
```

`git status`

Muestra el estado actual del repositorio.

`git add`

Agrega cambios al staging area.

`git commit`

Registra cambios en el historial.

`git log`

Muestra el historial de commits.

git diff

Muestra diferencias entre versiones de archivos.

```
git diff
```

git restore

Permite descartar cambios locales en archivos.

```
git restore archivo.v
```

git rm

Elimina archivos del repositorio.

```
git rm archivo.txt
```

11.6. Ramas

Las ramas son una de las características más poderosas de Git. Supóngase que el proyecto funciona correctamente y se desea agregar una nueva funcionalidad. En lugar de modificar directamente la rama principal, se crea una rama nueva para aislar el trabajo.

Crear una rama

```
git branch feature/alu
```

Cambiar de rama

```
git checkout feature/alu
```

Crear y cambiar en un solo paso

En caso de que se quiera crear la rama y trasladarse a esta al mismo tiempo.

```
git checkout -b feature/alu
```

Listar ramas

```
git branch
```

Fusionar una rama

Se requiere volver a la rama principal (main) o a la que se quiere combinar y luego indicar cuál rama se quiere unir a la actual:

```
git switch main
git merge feature/alu
```

11.7. Conflictos de fusión

Cuando se unen varias ramas, muchos cambios pueden integrarse fácilmente y Git va a tratar de mantener un orden. Sin embargo, muchas veces se edita un mismo archivo y esto puede complicar la tarea de unir archivos, ya que se desconoce el orden en el que se deben poner los cambios, generando conflictos.

Un conflicto ocurre cuando Git no puede decidir automáticamente cómo combinar cambios de dos ramas. Esto sucede, por ejemplo, cuando dos personas modifican la misma línea de un archivo.

En un conflicto, Git marca el archivo para que el desarrollador decida qué versión conservar. Por ejemplo:

```
<<<<<< HEAD
assign Y = A & B;
=====
assign Y = A | B;
>>>>>> feature/or_logic
```

El desarrollador debe:

1. editar manualmente el archivo
2. dejar la versión correcta
3. agregar el archivo nuevamente con `git add`
4. completar la fusión con un commit

11.8. Repositorios remotos

Git permite sincronizar el trabajo local con servidores remotos.

Agregar remoto

```
git remote add origin https://github.com/usuario/proyecto.git
```

Enviar cambios

```
git push origin main
```

Obtener cambios

```
git pull origin main
```

Diferencia entre fetch y pull

- `git fetch`: descarga cambios remotos sin integrarlos
- `git pull`: descarga e integra cambios automáticamente

Es importante destacar que, antes de iniciar a trabajar en un proyecto, debe descargarse la última versión de la rama. Asimismo, al concluir el trabajo, es buena práctica enviar los cambios al repositorio remoto. Esto ahorra problemas como pérdidas de avance con el avance de un proyecto, vacaciones de un colaborador, entre otros.

11.9. Archivos que no deben versionarse

No todos los archivos del proyecto deben incluirse en Git, ya que son propios de una ejecución. En proyectos de diseño digital normalmente conviene excluir:

- archivos temporales
- resultados de compilación
- bitstreams generados
- archivos intermedios de simulación
- cachés de herramientas
- archivos dependientes del sistema operativo

Para ello se utiliza el archivo `.gitignore`. Ejemplo:

```
*.jou  
*.log  
*.str  
*.tmp  
*.bit  
*.cache/  
*.runs/  
*.Xil/
```

11.10. Buenas prácticas generales con Git

Convencionalidad en commits: Conventional Commits

Es preferible realizar commits pequeños y coherentes en lugar de commits grandes y ambiguos.

Un buen commit debe representar una sola intención, por ejemplo:

- agregar un módulo
- corregir un error
- actualizar documentación

Conventional Commits es una convención para estandarizar mensajes de commit. El mensaje del commit debe explicar qué se hizo.

Su formato básico es:

```
tipo(alcance): descripcion
```

Ejemplos:

```
feat(alu): agrega operacion xor  
fix(counter): corrige overflow en reset  
docs(readme): actualiza instrucciones de compilacion  
refactor(tb): reorganiza generacion de estímulos  
test(fsm): agrega casos para estado de error
```

Tipos comunes

- **feat**: nueva funcionalidad
- **fix**: corrección de error
- **docs**: documentación
- **style**: cambios de formato sin alterar funcionalidad
- **refactor**: reestructuración interna
- **test**: pruebas
- **chore**: tareas auxiliares o mantenimiento

Versionado semántico

El **Semantic Versioning** es una convención para numerar versiones de un proyecto de forma significativa.

El formato es:

MAJOR.MINOR.PATCH

Por ejemplo:

- 1.0.0
- 1.2.0
- 2.0.3

Significado de cada número

- **MAJOR**: cambia cuando hay incompatibilidades importantes
- **MINOR**: cambia cuando se agregan funcionalidades compatibles
- **PATCH**: cambia cuando se corrigen errores sin romper compatibilidad

Ejemplos

- pasar de 1.0.0 a 1.0.1: corrección pequeña
- pasar de 1.0.0 a 1.1.0: nueva funcionalidad compatible
- pasar de 1.0.0 a 2.0.0: cambio incompatible

En proyectos de diseño digital, Semantic Versioning puede aplicarse a IP cores y bibliotecas HDL.

GitFlow

GitFlow es una estrategia de trabajo basada en ramas con roles bien definidos. Aunque existen flujos más simples, GitFlow es útil pedagógicamente porque ayuda a entender la organización del trabajo en proyectos estructurados.

Ramas principales

- **main**: contiene versiones estables y liberadas. Tiene calidad de producción
- **develop**: integra el desarrollo en curso o un sistema que aún no tiene suficientes pruebas como para considerarse en calidad de producción.

Ramas auxiliares

- **feature/***: nuevas funcionalidades
- **release/***: preparación de una versión
- **hotfix/***: corrección urgente sobre producción

Flujo típico

1. Se crea una rama **feature** desde **develop**
2. Se desarrolla la funcionalidad
3. Se fusiona de nuevo en **develop**
4. Cuando el sistema está listo, se crea una rama **release**
5. La rama **release** se valida y corrige
6. Se fusiona en **main** y **develop**
7. Si aparece un error crítico en producción, se crea una rama **hotfix** desde **main**

GitFlow propone una estructura clara, buena separación entre desarrollo y liberación y es útil para equipos y entregas planificadas. Sin embargo, puede ser complejo para proyectos pequeños, ya que introduce más ramas y ceremonias y puede ser excesivo.

Esto implica que GitFlow resulta útil cuando existen varias personas desarrollando en paralelo, hay entregas formales y se manejan versiones estables y versiones en desarrollo.

Flujo simple recomendado para principiantes

Para estudiantes que comienzan, un flujo sencillo suele ser más apropiado:

1. partir de **main**
2. crear una rama por tarea
3. trabajar en commits pequeños
4. fusionar de vuelta cuando la tarea esté validada

Ejemplo:

```
git switch main
git pull origin main
git switch -c feature/testbench_mux
# editar archivos
git add .
```

```
git commit -m "feat(tb): agrega testbench para mux2"  
git switch main  
git merge feature/testbench_mux  
git push origin main
```

Etiquetas y liberaciones

Git permite etiquetar commits importantes, por ejemplo para marcar versiones.

Crear una etiqueta

```
git tag v1.0.0
```

Ver etiquetas

```
git tag
```

Enviar etiquetas al remoto

```
git push origin v1.0.0
```

Las etiquetas son especialmente útiles al combinar Git con Semantic Versioning.

Estructura recomendada de un repositorio

Un repositorio ordenado mejora mucho la colaboración. Ejemplo para un proyecto de diseño digital:

```
proyecto/  
  rtl/  
  tb/  
  scripts/  
  constraints/  
  docs/  
  sim/  
  README.md  
  .gitignore
```

Donde `rtl/` es el código HDL sintetizable, `tb/` bancos de prueba, `scripts/` scripts Bash, Tcl o Python, `constraints/` archivos XDC u otros, `docs/` documentación y `sim/`: archivos relacionados con simulación.

README y documentación del repositorio

Todo repositorio debe incluir un archivo `README.md` con al menos:

- descripción del proyecto
- estructura de carpetas
- instrucciones de compilación o simulación
- dependencias
- autoría o integrantes

11.11. Errores comunes

Usar Git como respaldo desordenado

Git no debe usarse simplemente como un lugar donde se suben archivos sin criterio. El historial debe ser comprensible.

Hacer commits demasiado grandes

Cuando un commit cambia decenas de archivos sin una intención clara, se dificulta revisar y depurar.

No usar `.gitignore`

Esto puede llenar el repositorio con archivos temporales innecesarios.

Trabajar directamente siempre en main

Esto aumenta el riesgo de introducir errores en la rama principal.

Mensajes como “update” o “fix”

Estos mensajes no describen realmente qué cambió.

No revisar `git status`

Muchos errores se evitan simplemente revisando el estado antes de agregar o committear cambios.

11.12. Resumen

Git es una herramienta fundamental para controlar versiones de proyectos técnicos y colaborar de forma ordenada.

El dominio de conceptos como repositorio, commit, ramas y merge permite desarrollar proyectos de manera mucho más segura y profesional.

Además, prácticas como GitFlow, Conventional Commits y Semantic Versioning ayudan a estructurar el trabajo, mejorar la trazabilidad y mantener un historial de cambios claro y útil.

12.1. Comunicación UART

Introducción

UART (*Universal Asynchronous Receiver-Transmitter*) es uno de los protocolos de comunicación serial más utilizados en sistemas digitales. Se emplea ampliamente para la comunicación entre microcontroladores, FPGAs, computadoras y dispositivos periféricos.

UART es un protocolo **asíncrono**, lo que significa que no utiliza una señal de reloj compartida entre transmisor y receptor. En su lugar, ambos dispositivos deben configurarse previamente con la misma velocidad de transmisión (baud rate).

Las características principales del protocolo UART son su comunicación serial (bit a bit) asincrónica, lo que significa que cada bit se envía por cada ciclo de reloj, mediante dos líneas de transmisión: TX (transmisión), RX (recepción). Otra cosa que destaca es que tiene una configuración simple, donde se pueden añadir métodos de detección de errores como bits de paridad y bits de sincronización, que son usualmente 2 bits (inicio y parada).

Formato de trama UART

La comunicación UART se realiza mediante tramas de datos con una estructura específica.

Start	D0	D1	D2	D3	D4	D5	D6	D7	Stop
-------	----	----	----	----	----	----	----	----	------

Una trama típica incluye:

- 1 bit de inicio (Start bit)
- 8 bits de datos
- opcionalmente un bit de paridad
- 1 o más bits de parada (Stop bits)

En cuanto a los niveles de señal, se define a la línea en reposo cuando se encuentra en un 1. Cuando se inicia la transmisión, el bit de inicio se define como 0, mientras que el de fin de trama es de 1.

Ejemplo de transmisión

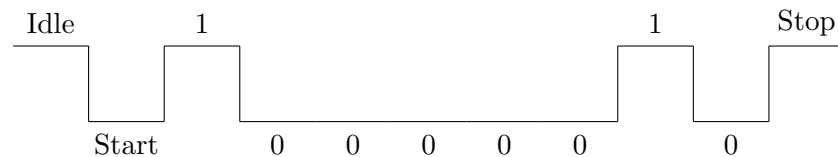
Para transmitir el valor binario:

01000001 (*ASCII : 'A'*)

La secuencia UART sería:

Start $\rightarrow$ 1 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 0 $\rightarrow$ 1 $\rightarrow$ 0 $\rightarrow$ Stop

Diagrama temporal



Baud rate

El baud rate define la velocidad de transmisión:

$$\text{Baud Rate} = \frac{1}{T_{bit}}$$

Ejemplos comunes:

- 9600 bps
- 115200 bps

En sistemas FPGA, se utiliza un divisor de reloj para generar el tiempo de bit adecuado.

Ejemplo de transmisor UART en Verilog

```
module uart_tx (  
    input wire clk,  
    input wire start,  
    input wire [7:0] data,  
    output reg tx = 1  
);  
  
reg [3:0] bit_count;  
reg [9:0] shift_reg;  
  
always @(posedge clk)  
begin  
    if (start) begin  
        shift_reg <= {1'b1, data, 1'b0};  
        tx <= 1;  
    end else begin  
        tx <= shift_reg[0];  
        shift_reg <= shift_reg >> 1;  
        shift_reg[8] <= 1;  
    end  
end  
  
endmodule
```

Aplicaciones

UART es ampliamente utilizado en comunicación con computadoras (puerto serial / USB-UART), depuración de sistemas embebidos, transmisión de datos entre microcontroladores y configuración de dispositivos FPGA.

Dentro de las ventajas se encuentran su implementación sencilla, bajo costo y la no transmisión de reloj. Sin embargo, posee menor velocidad comparada con SPI o Ethernet, no incluye sincronización explícita y es susceptible a errores si no hay buena configuración.

12.2. Comunicación SPI

Introducción

SPI (*Serial Peripheral Interface*) es un protocolo de comunicación serial síncrono ampliamente utilizado en sistemas digitales para la comunicación entre dispositivos como microcontroladores, FPGAs, memorias y sensores.

A diferencia de UART, SPI utiliza una señal de reloj compartida, lo que permite una comunicación más rápida y sincronizada.

SPI sigue una arquitectura maestro-esclavo (o servidor-cliente), donde el maestro propone el reloj y maneja la selección del cliente al cual va a escribir o recibir información. Asimismo, a diferencia de UART, SPI es un protocolo con alta velocidad de transmisión y comunicación full-duplex (que recibe y transmite al mismo tiempo).

Señales de SPI

SPI utiliza típicamente cuatro señales principales:

- **SCLK** (Serial Clock): señal de reloj generada por el maestro
- **MOSI** (Master Out Slave In): datos del maestro al esclavo
- **MISO** (Master In Slave Out): datos del esclavo al maestro
- **SS** o **CS** (Slave Select / Chip Select): selección del dispositivo esclavo

Diagrama de conexión

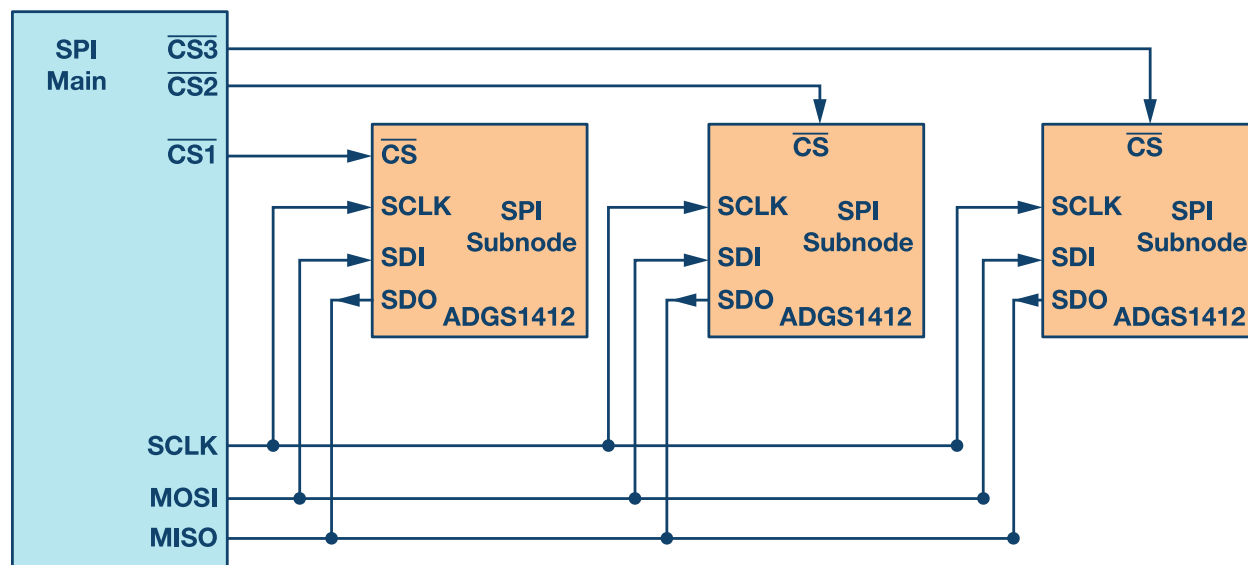


Figura 12.1: Diagrama de conexión de SPI con múltiples clientes. Obtenido de Analog Devices.

Funcionamiento

El maestro controla la comunicación generando la señal de reloj (SCLK), seleccionando el esclavo mediante CS y enviando y recibiendo datos simultáneamente.

La transmisión ocurre bit a bit sincronizada con el reloj.

Diagrama temporal

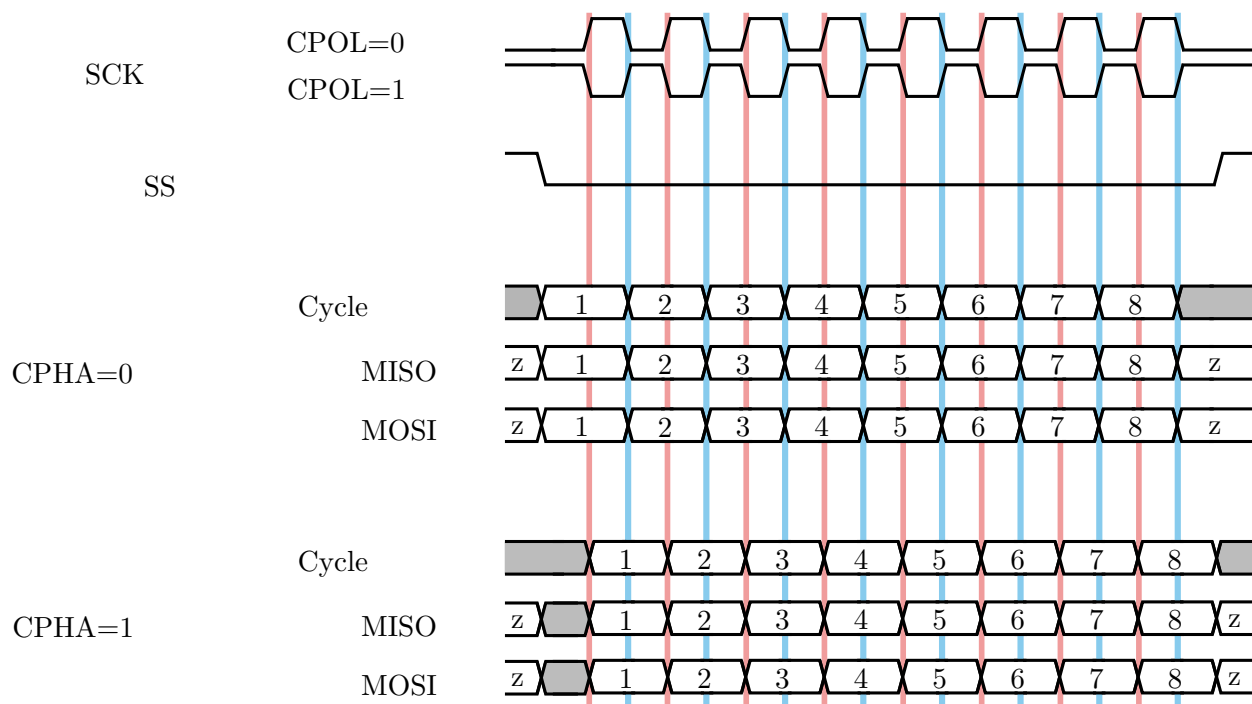


Figura 12.2: Diagrama de tiempos de SPI. Obtenido de Analog Devices.

SPI define cuatro modos de operación basados en dos parámetros:

- **CPOL** (Clock Polarity)
- **CPHA** (Clock Phase)

Modo	CPOL	CPHA
0	0	0
1	0	1
2	1	0
3	1	1

Estos modos determinan cuándo se muestrean los datos y cuándo se actualizan los datos.

Durante una transferencia SPI, el maestro envía un bit por MOSI o el esclavo envía un bit por MISO. También es posible que ambos intercambien datos simultáneamente, haciendo que SPI sea **full-duplex**.

Ejemplo conceptual en Verilog

```
always @(posedge clk) begin
    if (sclk_rising_edge) begin
        shift_reg <= {shift_reg[6:0], miso};
    end

    if (sclk_falling_edge) begin
        mosi <= shift_reg[7];
    end
end
```

Este ejemplo muestra cómo se desplazan datos en cada ciclo de reloj y como se sincroniza la transmisión con SCLK.

Aplicaciones

SPI es ampliamente utilizado en memorias flash, sensores digitales con alta velocidad, pantallas (LCD, OLED), convertidores ADC/DAC de gran resolución, comunicación entre FPGA y periféricos de velocidad, esto ya que el SPI posee alta velocidad (decenas de Mbps), comunicación full-duplex sincrónica e implementación simple en hardware. Sin embargo, requiere más líneas que UART, no tiene direccionamiento interno (usa líneas SS) y el estándar no es rígido (variaciones entre dispositivos).

12.3. Comunicación I2C

Introducción

I2C (*Inter-Integrated Circuit*) es un protocolo de comunicación serial síncrono ampliamente utilizado para la interconexión de circuitos integrados en sistemas digitales.

Fue desarrollado para permitir la comunicación entre múltiples dispositivos utilizando un número mínimo de líneas. A diferencia de SPI, I2C utiliza únicamente dos señales: SDA (*Serial Data*) y SCL (*Serial Clock*).

Dentro de las características principales, se encuentra la comunicación síncrona. Posee una arquitectura maestro-esclavo, como en el SPI, y tiene soporte para múltiples dispositivos de la misma manera en que el SPI lo hace o mediante registros de identificación (direcciones). Sin embargo, la comunicación half-duplex (se recibe o se transmite, pero no ambas).

Señales del bus

- **SCL**: señal de reloj generada por el maestro
- **SDA**: línea bidireccional de datos

Ambas líneas utilizan configuración **open-drain** y requieren resistencias pull-up externas.

Diagrama de conexión

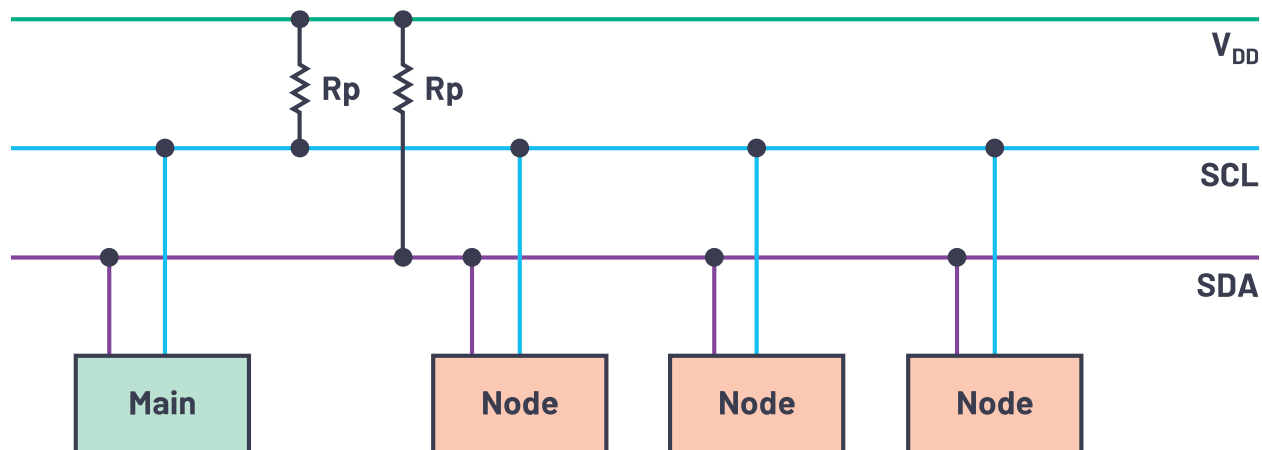


Figura 12.3: Diagrama de conexión de I2C con múltiples clientes. Obtenido de Analog Devices.

En cuanto a las señales, estas tienen algunas condiciones especiales como parte del estándar del protocolo, las cuales son:

Condición de inicio (Start): Se produce cuando SDA pasa de alto a bajo mientras SCL está en alto. Esto se ilustra en la Figura 12.4.

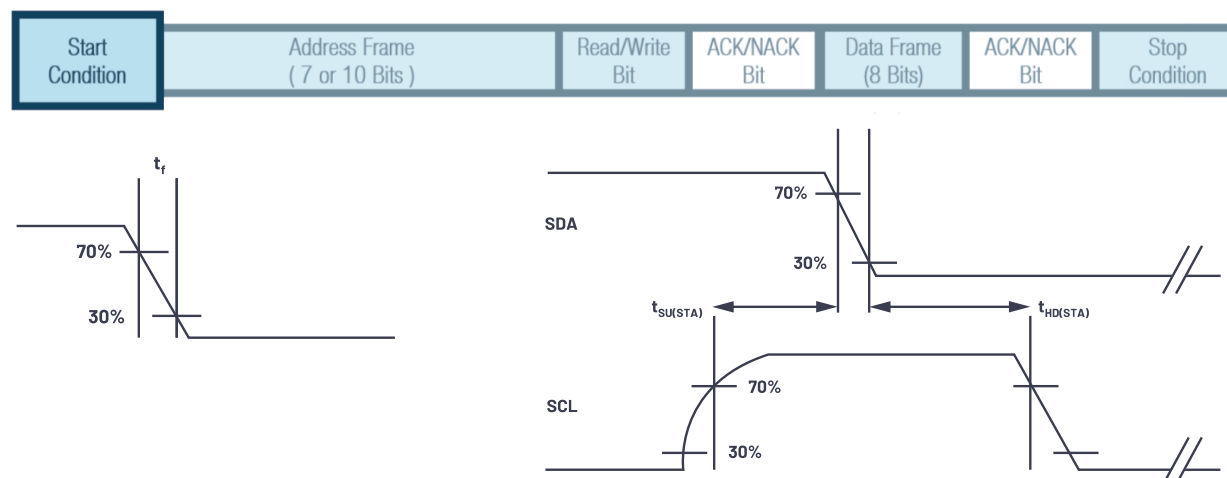


Figura 12.4: Ilustración del start. Esto es análogo para el stop. Tomado de Analog Devices.

Condición de parada (Stop): Se produce cuando SDA pasa de bajo a alto mientras SCL está en alto.

Caso de escritura (write):

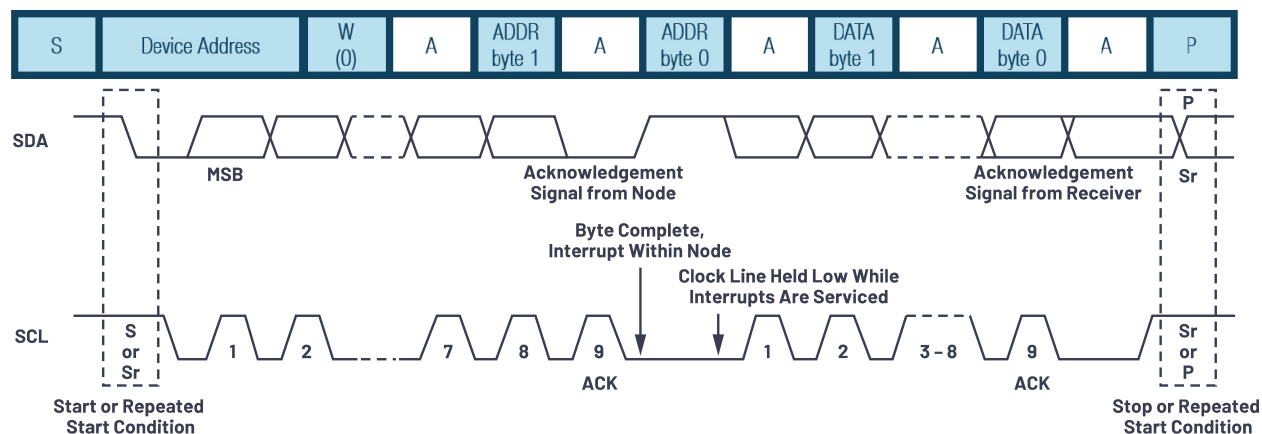


Figura 12.5: Ilustración del write. Tomado de Analog Devices.

Figura 12.6: Ilustración del read. Tomado de Analog Devices.

Una transferencia I2C típica incluye:

1. Condición de inicio
2. Dirección del dispositivo (7 bits)
3. Bit de lectura/escritura (R/W)
4. Bit de reconocimiento (ACK)
5. Datos (8 bits)
6. ACK
7. Condición de parada

El bit de ACK (Acknowledgment) se activa después de cada byte transmitido, donde el receptor debe responder con un bit ACK (0). Si no responde (NACK), la comunicación puede finalizar.

El funcionamiento de la comunicación comienza con el maestro iniciando la comunicación, generando una condición Start, enviando la dirección del esclavo e indicando lectura o escritura.

Luego, el esclavo responde con ACK si reconoce la dirección. En caso contrario, hace NACK. Si hay un ACK, se transfieren datos byte a byte y cada byte es confirmado con ACK. La comunicación termina con Stop.

Ejemplo conceptual

Escritura a un dispositivo:

1. Start
2. Dirección + W
3. ACK
4. Byte de datos
5. ACK
6. Stop

Velocidad

I2C soporta varias velocidades:

- Standard: 100 kHz
- Fast: 400 kHz
- Fast+: 1 MHz
- High-speed: 3.4 MHz

Ejemplo conceptual en Verilog

```
always @(posedge clk) begin
    case (state)
        START: sda <= 0;
        ADDRESS: shift_reg <= {addr, rw};
        DATA: shift_reg <= data;
        STOP: sda <= 1;
    endcase
end
```

Aplicaciones

I2C es ampliamente utilizado en sensores (temperatura, acelerómetros) con velocidad media, memorias EEPROM, RTC (relojes en tiempo real), configuraciones de dispositivos como cámaras de vídeo y comunicación entre chips en una placa.

Dentro de las ventajas se encuentran que la comunicación física solo requiere dos líneas, tiene soporte para múltiples dispositivos e incluye direccionamiento integrado. Sin embargo, tiene una menor velocidad que SPI, mayor complejidad de protocolo y requiere resistencias pull-up

12.4. Comunicación USB

Introducción

USB (*Universal Serial Bus*) es un estándar de comunicación serial ampliamente utilizado para la conexión de dispositivos electrónicos a computadoras. A diferencia de protocolos como UART, SPI o I2C, USB es un protocolo mucho más complejo que incluye múltiples capas, manejo de dispositivos, control de errores y negociación automática.

USB permite conectar una gran variedad de dispositivos como teclados y ratones, memorias flash, dispositivos de comunicación e interfaces de desarrollo (USB-UART, USB-JTAG).

Se caracteriza por ser una comunicación serial de alta velocidad, incluyendo una arquitectura host-dispositivo, soporte para múltiples dispositivos, configuración automática (plug and play) y transferencias confiables con control de errores.

USB sigue un modelo **host-dispositivo**, donde el host controla toda la comunicación y los dispositivos responden a solicitudes del host.

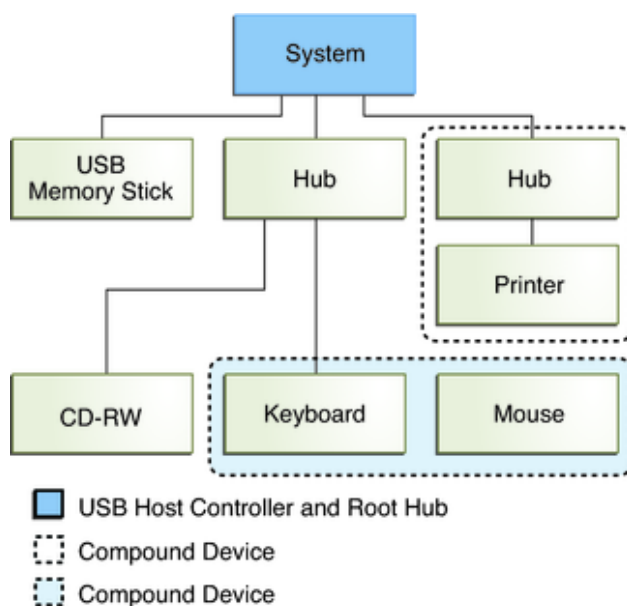


Figura 12.7: Ilustración de la arquitectura host de USB. Tomado de Oracle.

USB permite conectar múltiples dispositivos mediante hubs, tal como ilustra la Figura 12.7, ampliando la capacidad de conexión de dispositivos.

A nivel físico, el USB utiliza dos líneas principales de datos que son un par diferencial (D+, D-), lo que mejora la relación señal a ruido, mediante la cancelación del fenómeno de crosstalk.

Figura 12.8: Ilustración del conector USB. Tomando de Media Common.

Velocidades de operación

USB soporta múltiples velocidades:

- Low Speed (USB 1.0): 1.5 Mbps
- Full Speed (USB 1.1): 12 Mbps
- High Speed (USB 2.0): 480 Mbps
- SuperSpeed (USB 3.x): 5 Gbps y superiores

Capas del protocolo

USB se organiza en varias capas:

- **Capa física:** transmisión eléctrica de señales
- **Capa de enlace:** manejo de paquetes y errores
- **Capa de protocolo:** estructura de transacciones
- **Capa de aplicación:** interacción con el dispositivo

Tipos de transferencia

USB define distintos tipos de transferencia según la aplicación, dependiendo de la tarea o de la cantidad de datos a enviar. Por ejemplo, se tiene el tipo **control**, que ayuda en la configuración del dispositivo, el **bulk**, que permite transferencia de grandes cantidades de datos, **interrupt** para comunicación periódica (ej. teclado) y el **isochronous**: datos en tiempo real (audio/video).

La comunicación USB se basa en transacciones organizadas en (1) Token (indica la operación), (2) data (datos transmitidos) y (3) handshake (confirmación):

Token → Data → ACK

Cuando un dispositivo USB se conecta, el host detecta el dispositivo, le asigna una dirección, identifica el tipo de dispositivo y carga dinámicamente el controlador adecuado. A este proceso se denomina **enumeración**.

Aplicación en sistemas digitales

En FPGA, normalmente no se implementa USB desde cero debido a su complejidad. En su lugar, se utilizan convertidores USB-UART, controladores USB externos y IP cores especializados proveídos por el vendedor o fabricante.

Dentro de las ventajas de USB está su alta velocidad, conexión universal, soporte para múltiples dispositivos y configuración automática. Sin embargo, oculta una alta complejidad de implementación, requiere hardware especializado y es difícil de implementar desde cero en FPGA.

Comparación con otros protocolos

Característica	UART	SPI	I2C	USB
Complejidad	Baja	Media	Media	Alta
Velocidad	Baja	Alta	Media	Muy alta
Arquitectura	Punto a punto	Maestro-esclavo	Multi-dispositivo	Host-dispositivo
Implementación FPGA	Fácil	Media	Media	Difícil

12.5. Protocolo AXI

Introducción

AXI (*Advanced eXtensible Interface*) es un protocolo de comunicación de alto rendimiento definido dentro del estándar AMBA (Advanced Microcontroller Bus Architecture) desarrollado por ARM. AXI es ampliamente utilizado en sistemas digitales complejos, incluyendo SoCs (System on Chip), FPGAs modernas (Xilinx/AMD, Intel), procesadores embebidos y aceleradores de hardware. Es un protocolo ampliamente usado por tener un alto ancho de banda y *throughput* (B/s).

A diferencia de protocolos como UART, SPI o I2C, AXI está diseñado para comunicación de alta velocidad **interna** dentro de un sistema, no para comunicación externa. Asimismo, es una comunicación paralela.

Sus principales características son la comunicación síncrona, arquitectura maestro-esclavo, alta velocidad y alto throughput y soporte para transacciones en paralelo (*full-duplex*) y el uso de canales independientes.

Arquitectura

El protocolo AXI maneja sus transacciones a través de los siguientes canales de comunicación:

- *Write request*: que tiene como prefijo AW en el nombre de las señales. Permite iniciar una transacción de escritura.

- *Read request*: que tiene como prefijo AR en el nombre de las señales. Permite iniciar una transacción de lectura.
- *Write data*: que tiene como prefijo W en el nombre de las señales. Porta los datos que se van a escribir.
- *Read data*: que tiene como prefijo R en el nombre de las señales. Por los datos que se van a leer.
- *Write response*: que tiene como prefijo B en el nombre de las señales. Es la respuesta de que el receptor recibió el mensaje.

En cada transacción, se manejan direcciones y los dispositivos se ven como bancos de registros o dispositivos mapeados a memoria. También soporta el completado de transacciones fuera de orden.

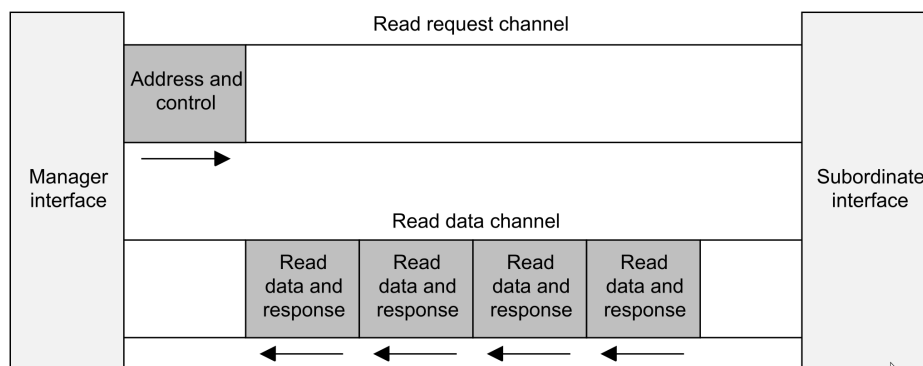


Figura 12.9: Arquitectura de los canales de lectura. Obtenido de ARM.

Las Figuras 12.9 y 12.10 muestran el proceso de intercambio de datos desde el alto nivel. Es importante destacar que cada parte horizontal es un canal de comunicación con conexiones físicas (Figura 12.11). Es decir, el bus de direcciones de escritura es distinto que el de lectura, aplicable también a los canales de datos.

Transacciones

Canales de write and read requests

Hay canales de *request* separados para lectura y escritura. Estos tienen el paquete de control que incluye la dirección de la transacción, la cantidad de bytes, entre otras posibles señales de control.

Canal de datos de escritura y lectura

Acarrea los datos desde el maestro hasta el subordinado. El canal de escritura puede tener un ancho de hasta 1024 bits y hay una señal de enmascarado cada byte, lo que permite indicar cuáles bytes dentro de dichos 1024 bits son válidos o si es un paquete de menor tamaño. Asimismo, la información siempre es tratada como encolada (*buffered*), por lo que el maestro puede escribir sin

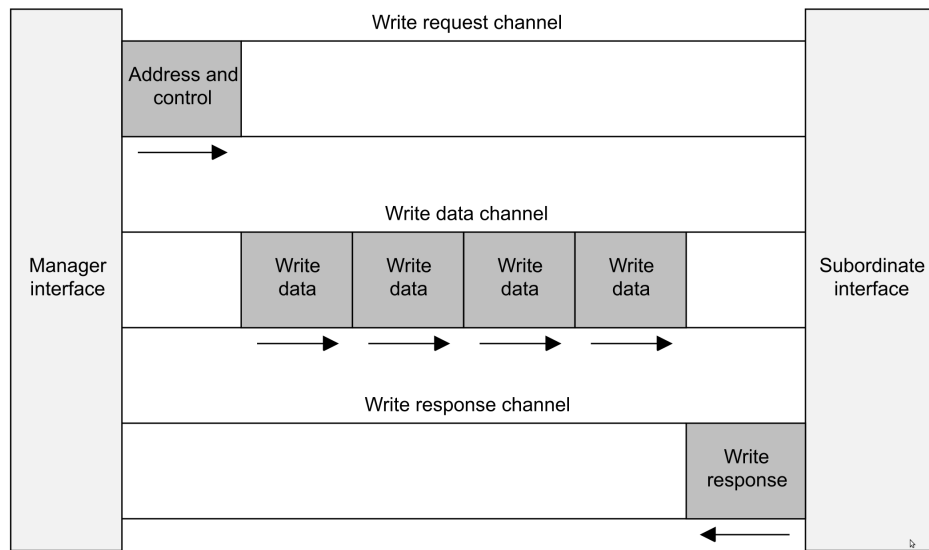


Figura 12.10: Arquitectura de los canales de escritura. Obtenido de ARM.

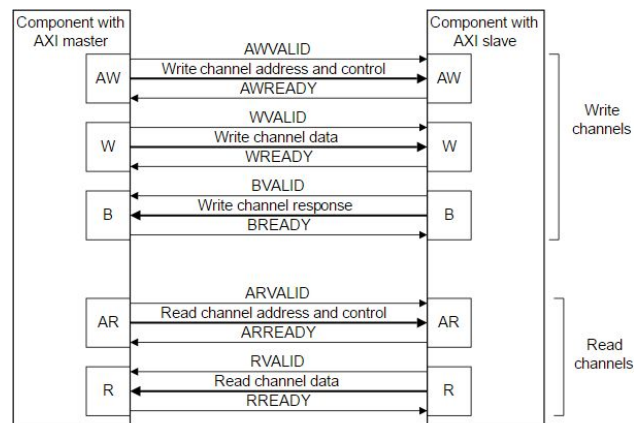


Figura 12.11: AXI4 channels. Obtenido de FPGA Emu.

respuesta de reconocimiento (acknowledgement) por parte del subordinado.

Esto es homólogo para el canal de lectura.

Canal de respuesta de escritura

Contiene señales de reconocimiento de la escritura por parte del subordinado.

Las transacciones son mostradas en las Figuras 12.12 y 12.13.

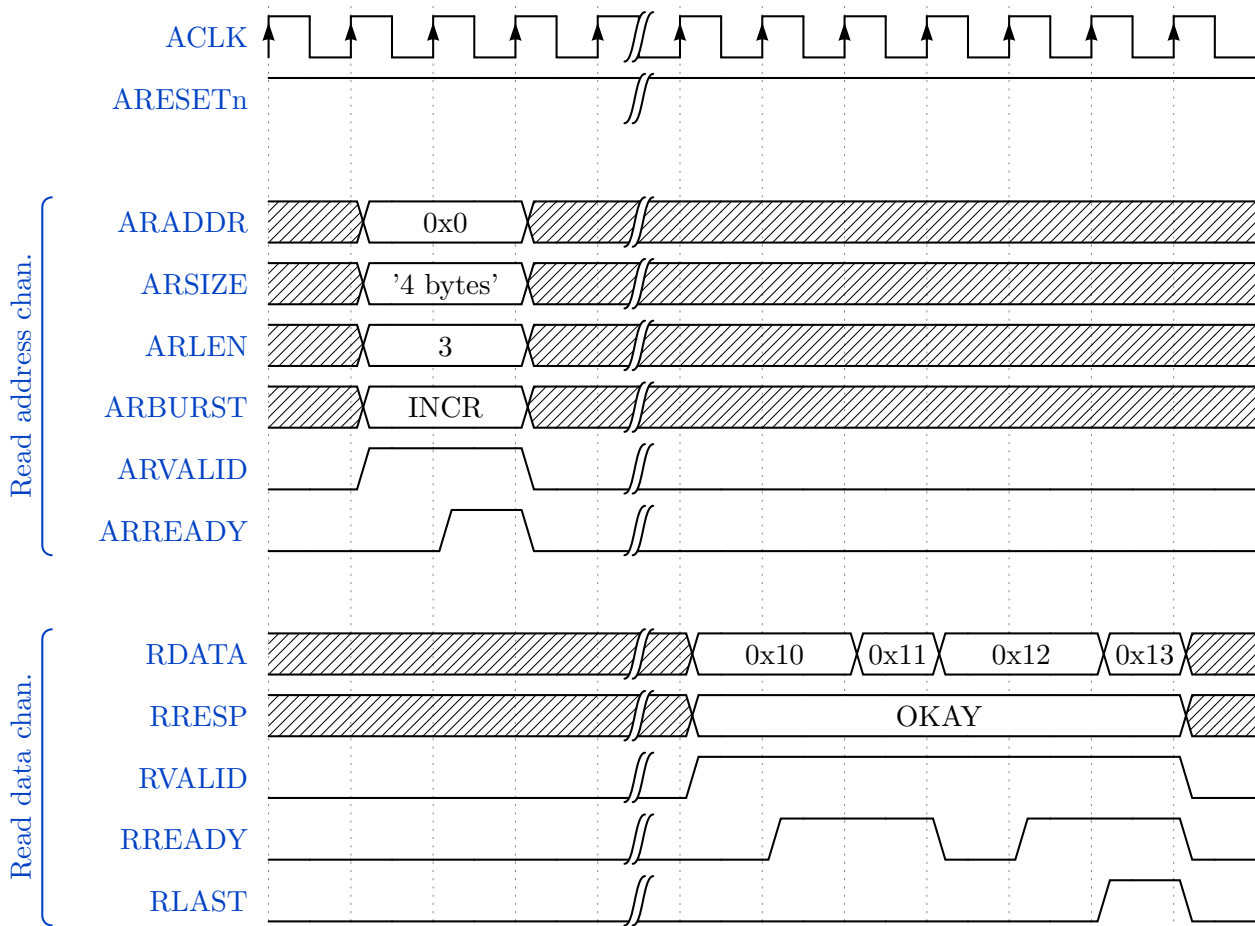


Figura 12.12: AXI4 read transaction. Obtenido de FPGA Emu.

Topologías de interconexión

La Figura 12.14 muestra una posible configuración de un sistema AXI. Se puede tener una interfaz arbitrada, cross-bar (ver Figura 12.15) o enrutada que puede contener múltiples maestros y subordinados. Generalmente, los sistemas tienen una interfaz que les permite conectarse al bus de AXI.

Una posible analogía es que el CPU puede ser un maestro, mientras que la RAM puede ser un subordinado, ya que puede escribirle a la RAM. Generalmente, el maestro se define como una interfaz capaz de escribir. Igualmente, es posible que un dispositivo o módulo pueda tener más de una interfaz, actuando como maestro o subordinado al mismo tiempo.

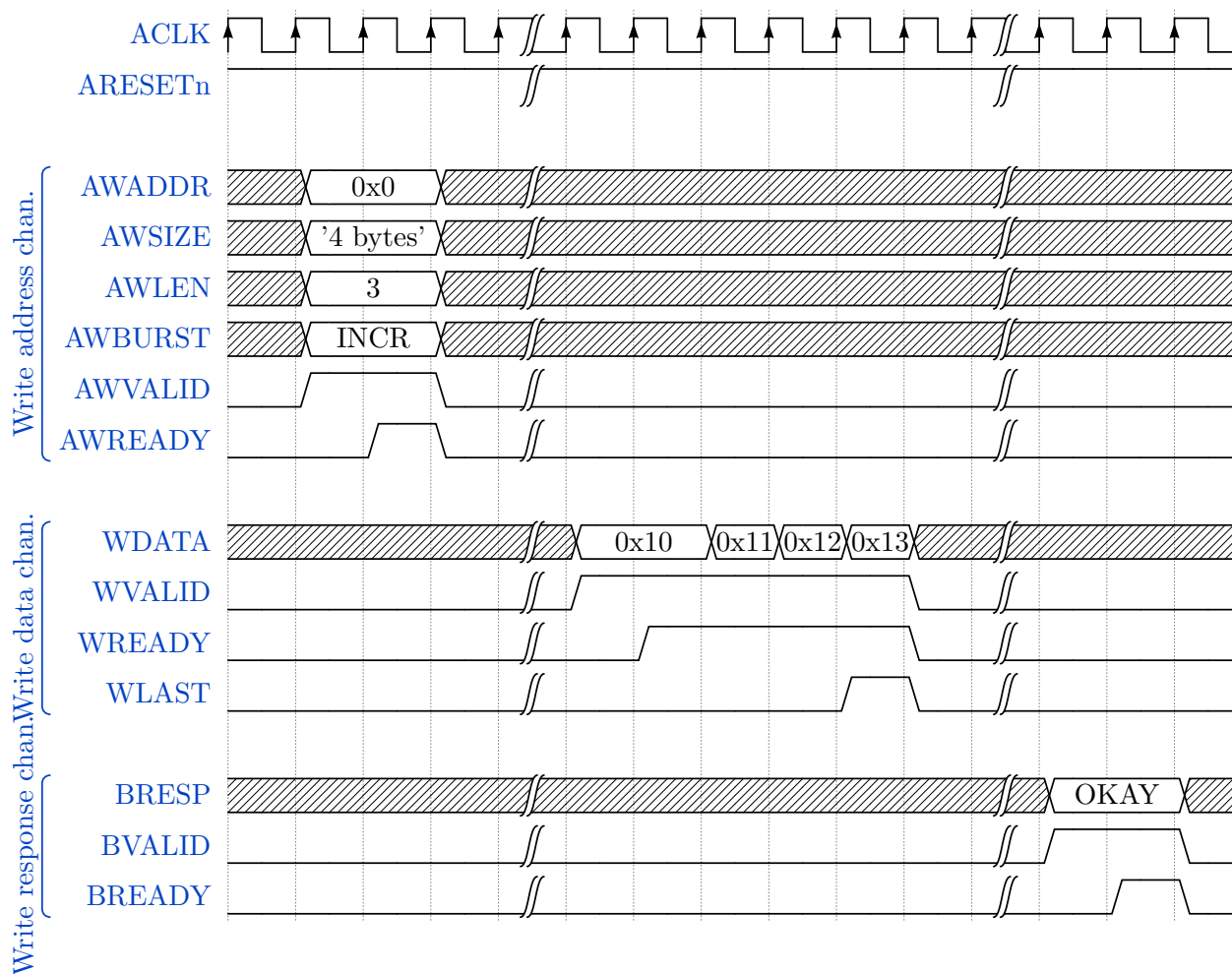


Figura 12.13: AXI4 write transaction. Obtenido de FPGA Emu.

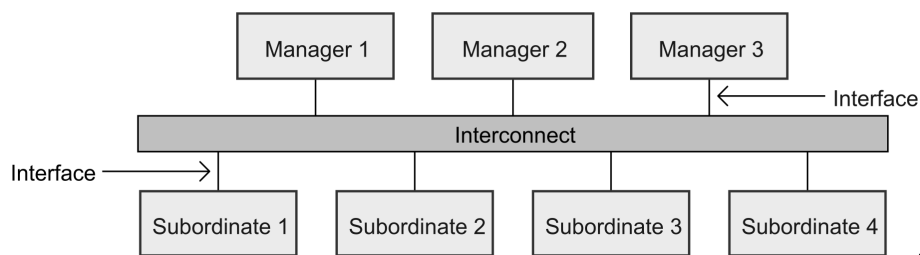


Figura 12.14: Interfaces e interconexiones. Obtenido de ARM.

Ejemplos de sistemas con AXI4

En este caso, el sistema se implementa en una AMD Kria KV260, donde el sistema de procesamiento (PS) se conecta a la lógica programable (PL) a través de un puerto AXI que se conecta a

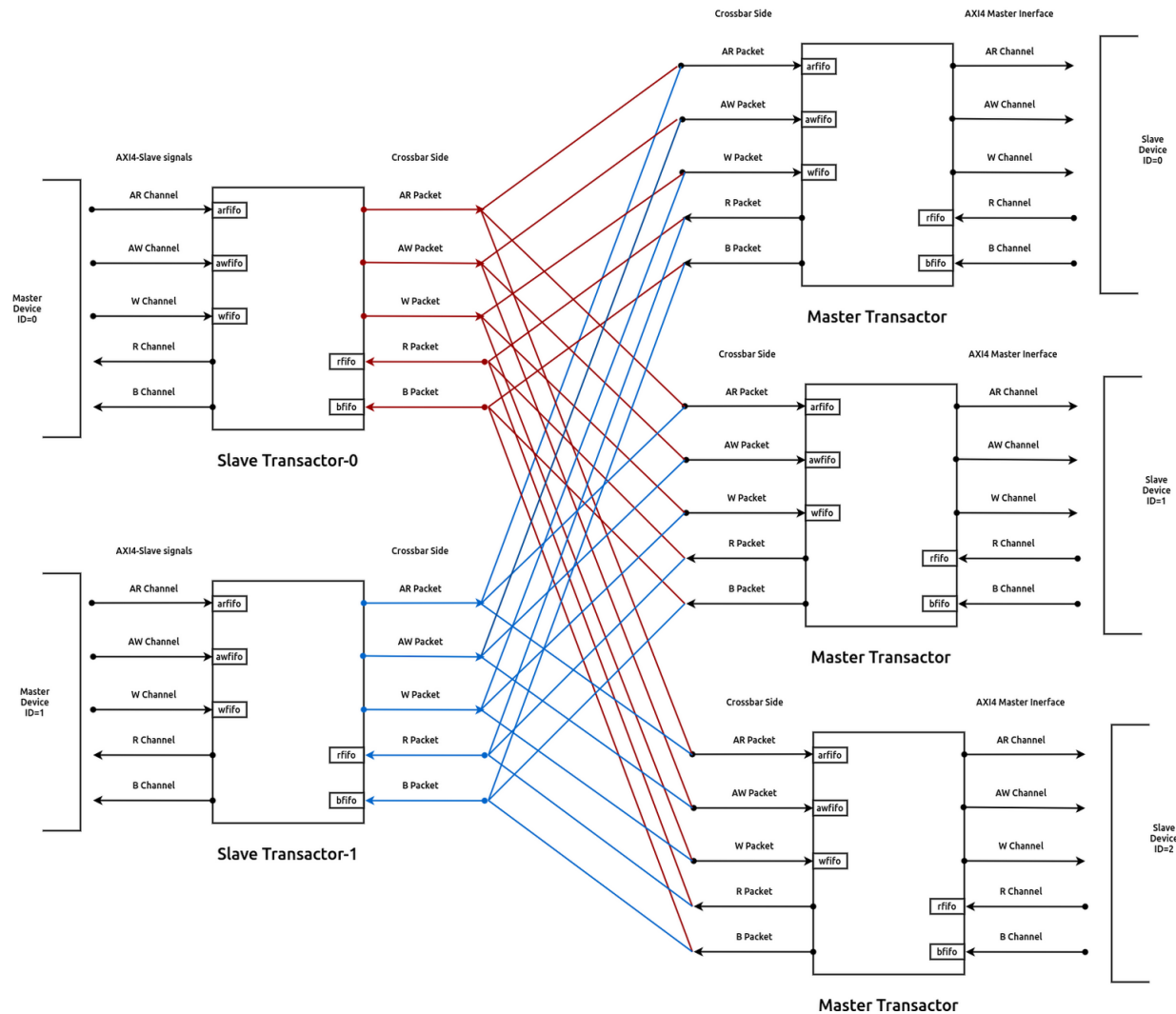


Figura 12.15: AXI Cross-bar. Obtenido de Fabrics.

un *interconnect*. Este contiene distintos subordinados, entre ellos un controlador SPI, una DMA y un GPIO, que luego se conectan a un sistema acelerado (ReckON).

Ventajas y desventajas

Dentro de las ventajas se encuentran su alta velocidad, gran flexibilidad, paralelismo y estándar ampliamente adoptado. Sin embargo, sus principales desventajas son su alta complejidad, gran número de señales y difícil implementación manual.

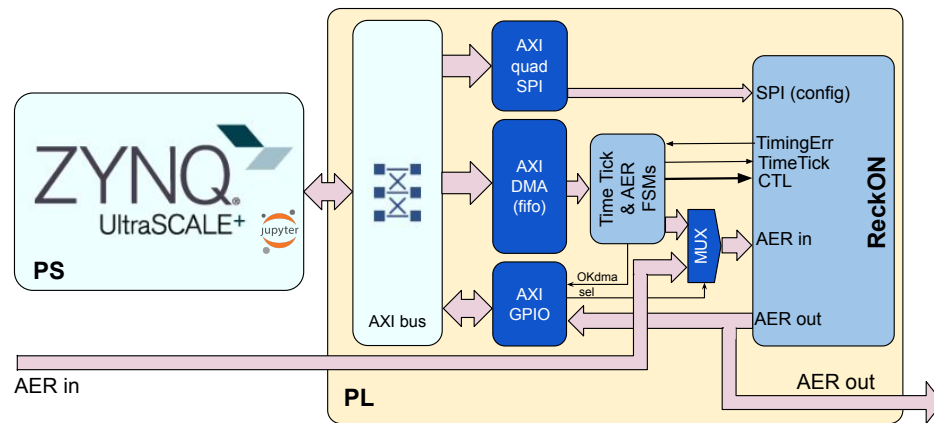


Figura 12.16: Ejemplo de un sistema basado en AXI4 en una AMD Kria KV260. Obtenido de ReckOn processor.

13.1. Introducción

En este capítulo se abordarán aspectos sobre el diseño de sistemas digitales en FPGA, tales como técnicas de optimización, selección de microarquitecturas, instanciación usando el diagrama de bloques y buses y las estrategias.

13.2. Síntesis de Memorias en FPGA

Las memorias son uno de los recursos más importantes en el diseño de sistemas digitales en FPGA. Dependiendo de cómo se describan en el código HDL, el sintetizador puede implementarlas utilizando diferentes tipos de recursos físicos disponibles en el dispositivo.

Entre los principales tipos de memoria en FPGA se encuentran:

- registros (flip-flops)
- LUTRAM (memoria distribuida)
- BRAM (Block RAM)
- URAM (Ultra RAM)

Comprender cómo inferir correctamente estos recursos desde código Verilog es fundamental para lograr diseños eficientes en términos de área, rendimiento y consumo de energía.

Concepto de inferencia de memoria

En HDL para FPGA, las memorias no siempre se instancian explícitamente. En muchos casos, el sintetizador **infiere** el tipo de memoria a partir del patrón de código. Esto significa que el diseñador describe el comportamiento y la herramienta decide la implementación física.

Factores que afectan la inferencia:

- tamaño de la memoria
- tipo de acceso (sincrónico/asíncrono)
- número de puertos
- estilo de codificación

Memoria basada en registros (Flip-Flops)

Las memorias pequeñas pueden implementarse utilizando flip-flops. Este enfoque es útil para registros como los de un banco de registros de procesador, buffers pequeños para FIFO y estructuras de control. Dentro de sus principales características se encuentran la rapidez y el alto consumo de recursos (los FF son usualmente de 1-bit), por lo que son recomendables para memorias pequeñas.

En Verilog, se puede hacer la inferencia de flip-flops mediante el siguiente código:

```
reg [7:0] mem [0:3];

always @(posedge clk) begin
    mem[addr] <= data_in;
end

assign data_out = mem[addr];
```

LUTRAM (Memoria distribuida)

Las LUTs de la FPGA pueden configurarse como pequeñas memorias. Esto se conoce como **LUTRAM** o memoria distribuida. Las LUTRAM son de baja latencia, pero son de tamaño limitado.

En Verilog, se puede hacer la inferencia mediante este código:

```
(* ram_style = "distributed" *) reg [7:0] mem [0:31];

always @(posedge clk) begin
    if (we)
```



```
        mem[addr] <= data_in;
    end

    assign data_out = mem[addr];
```

BRAM (Block RAM)

Las BRAM son bloques de memoria dedicados dentro de la FPGA, diseñados para almacenar grandes cantidades de datos. Poseen gran capacidad (alrededor de 18K bits - 36K bits), tienen acceso sincrónico, menor uso de LUTs y tienen soporte para dos puertos.

En Verilog, se puede hacer la inferencia mediante este código:

```
(* ram_style = "block" *) reg [7:0] mem [0:1023];

always @(posedge clk) begin
    if (we)
        mem[addr] <= data_in;

    data_out <= mem[addr];
end
```

URAM (Ultra RAM)

URAM es un tipo de memoria disponible en FPGAs de gama alta (por ejemplo, Xilinx UltraScale+), diseñada para grandes volúmenes de datos gracias a su alta capacidad. Sin embargo, tiene mayor latencia que las BRAM y son ideales para cachés de jerarquía superior.

Se puede controlar su inferencia mediante el siguiente código:

```
(* ram_style = "ultra" *) reg [31:0] mem [0:4095];
```

Memorias de doble puerto

Las memorias pueden permitir múltiples accesos simultáneos.

Ejemplo

```
reg [7:0] mem [0:255];

// Puerto A (escritura)
always @(posedge clk) begin
```

```

    if (we_a)
        mem[addr_a] <= data_a;
    end

    // Puerto B (lectura)
    always @(posedge clk) begin
        data_b <= mem[addr_b];
    end

```

Estas son utilizadas para FIFOs, búferes de comunicación y VRAM.

Comparación de tipos de memoria

Tipo	Tamaño	Velocidad	Recursos	Uso típico
FF	Muy pequeño	Muy alta (1 ciclo)	Alto	registros
LUTRAM	Pequeño	(1 2 ciclos)	Medio	tablas
BRAM	18K bits	(1 2 ciclos)	Bajo	buffers
URAM	288K bits	(2 3 ciclos)	Muy bajo	grandes datos

Buenas prácticas

- Usar LUTRAM o FF para memorias pequeñas
- Usar BRAM para buffers grandes
- Registrar salidas para mejorar timing
- Usar atributos cuando sea necesario
- Verificar la inferencia en reportes

Errores comunes

- Esperar BRAM con lectura asíncrona
- No registrar salidas
- Mezclar estilos de acceso
- No revisar reportes de síntesis

13.3. Síntesis de DSPs en FPGA

Las FPGAs modernas incluyen bloques dedicados para operaciones aritméticas llamados **DSP slices**. Estos bloques están optimizados para realizar operaciones como multiplicación, suma acumulativa y operaciones de punto fijo de manera eficiente. El uso adecuado de estos recursos permite mejorar el rendimiento, reducir el uso de LUTs y optimizar el consumo de energía, ya que la lógica digital se implementa de forma más densa y compacta.

Un DSP slice es un bloque especializado que típicamente incluye multiplicadores (por ejemplo, 18×25 bits), sumadores/acumuladores (48 bits), registros internos (pipeline) y lógica de control. Estos bloques están diseñados para aplicaciones como procesamiento digital de señales, inteligencia artificial, filtrado digital y operaciones matriciales.

Inferencia de DSPs

Al igual que las memorias, los DSPs pueden inferirse automáticamente a partir del código Verilog. El sintetizador reconoce patrones aritméticos y los implementa en DSP slices cuando es posible.

Multiplicación

Considere el siguiente patrón:

```
reg [15:0] A, B;  
wire [31:0] Y;  
  
assign Y = A * B;
```

Este patrón suele inferir un DSP automáticamente. Sin embargo, hay que considerar que el tamaño de los operandos afecta la inferencia y las multiplicaciones grandes pueden usar múltiples DSPs.

Suma y acumulación

Considere el siguiente patrón:

```
always @(posedge clk) begin  
    acc <= acc + data_in;  
end
```

Este patrón puede mapearse a un acumulador dentro de un DSP.

Multiplicación-acumulación (MAC)

Considere el siguiente patrón:

```
always @(posedge clk) begin
    acc <= acc + (A * B);
end
```

Este es uno de los patrones más importantes y típicamente se implementa completamente dentro de un DSP slice, por lo que se aprovecha la celda totalmente, reduciendo área y consumo de recursos.

Para mejorar el rendimiento, es recomendable usar registros intermedios.

```
reg [15:0] A_reg, B_reg;
reg [31:0] mult_reg;

always @(posedge clk) begin
    A_reg <= A;
    B_reg <= B;
    mult_reg <= A_reg * B_reg;
    Y <= mult_reg;
end
```

Asimismo, en herramientas como Vivado, se puede forzar el uso de DSPs:

```
(* use_dsp = "yes" *) wire [31:0] Y = A * B;
```

O evitar su uso:

```
(* use_dsp = "no" *) wire [31:0] Y = A * B;
```

Introducción al Lenguaje C para Sistemas Embebidos

14.1. Introducción

El lenguaje C es uno de los lenguajes más utilizados en sistemas embebidos debido a su eficiencia, cercanía al hardware y capacidad para manipular memoria de forma explícita. A diferencia de lenguajes de más alto nivel, C permite controlar directamente registros, periféricos, direcciones de memoria y estructuras de datos con bajo costo computacional.

En sistemas embebidos, C se utiliza para desarrollar firmware que se ejecuta sobre microcontroladores, procesadores embebidos o sistemas integrados en FPGA. Por esta razón, comprender su modelo de ejecución y memoria es fundamental.

14.2. Flujo de compilación y ejecución

Un programa en C no se ejecuta directamente desde el archivo fuente. Antes debe pasar por varias etapas de traducción, donde las etapas principales de la vida del código se define como:

Código C → Preprocesamiento → Compilación → Ensamblado → Enlazado → Ejecutable

Preprocesamiento

El código se preprocesa, donde el preprocesador interpreta directivas como includes, defines y pragmas como:

```
#include <stdint.h>
#define LED_PIN 5
```

En esta etapa se expanden macros y se insertan archivos de cabecera. Esto quiere decir que todos los defines se copian y pegan donde son invocados y los archivos incluidos se copian y pegan en un solo archivo junto con el código fuente, creando lo llamado *compilation units*.

Compilación

El compilador traduce el código C a código ensamblador específico de la arquitectura objetivo. Generalmente, el código C se convierte en un lenguaje intermedio, que es estándar entre compiladores de varios lenguajes y luego se genera el código ensamblador de la plataforma deseada.

Ensamblado

El ensamblador convierte el código ensamblador en código objeto.

Enlazado

El enlazador combina múltiples archivos objeto y bibliotecas para generar el programa final.

Ejecución en sistemas embebidos

En un computador convencional, el sistema operativo carga el programa. En un sistema embebido, el firmware normalmente se almacena en memoria no volátil, como Flash, y se ejecuta después del reinicio del sistema.

14.3. Constructs básicos del lenguaje

Variables y tipos de datos

C permite declarar variables con tipos explícitos:

```
int counter;  
char letter;  
float temperature;
```

En sistemas embebidos se recomienda usar tipos de tamaño fijo:

```
#include <stdint.h>  
  
uint8_t value8;  
uint16_t value16;  
uint32_t value32;  
int32_t signed_value;
```

Esto mejora la portabilidad y evita ambigüedades sobre el tamaño de los datos.

Constantes y macros

```
#define BUFFER_SIZE 128

const uint32_t baudrate = 115200;
```

Las macros son sustituciones realizadas por el preprocesador, mientras que `const` define una variable de solo lectura.

Operadores

C incluye operadores aritméticos, lógicos y bit a bit.

```
a + b
a - b
a & b
a | b
a ^ b
a << 1
a >> 1
```

En sistemas embebidos, los operadores bit a bit son esenciales para configurar registros.

Ejemplo: manipulación de bits

```
#define LED_MASK (1 << 3)

reg = reg | LED_MASK;    // Enciende bit
reg = reg & ~LED_MASK;   // Apaga bit
reg = reg ^ LED_MASK;    // Invierte bit
```

14.4. Flujo de programa

Condicionales

```
if (button_pressed) {
    led_on();
} else {
    led_off();
}
```

Switch

```
switch (state) {  
    case 0:  
        init_system();  
        break;  
  
    case 1:  
        run_system();  
        break;  
  
    default:  
        error_handler();  
        break;  
}
```

Ciclos

```
for (int i = 0; i < 10; i++) {  
    buffer[i] = 0;  
}
```

```
while (1) {  
    poll_inputs();  
    update_outputs();  
}
```

En sistemas embebidos, es común encontrar un ciclo infinito principal:

```
int main(void)  
{  
    system_init();  
  
    while (1) {  
        application_task();  
    }  
  
    return 0;  
}
```

14.5. Modelo de memoria

El modelo de memoria en C es fundamental para comprender cómo se almacenan y manipulan los datos.

Regiones típicas de memoria

Un programa embebido suele organizarse en varias regiones:

- **text**: contiene el código del programa
- **data**: variables globales inicializadas
- **bss**: variables globales no inicializadas
- **stack**: variables locales y llamadas a funciones
- **heap**: memoria dinámica

Variables globales

```
uint32_t counter = 0;  
uint8_t buffer[128];
```

Estas variables existen durante toda la ejecución del programa.

Variables locales

```
void function(void)  
{  
    uint32_t value = 10;  
}
```

Las variables locales normalmente se almacenan en el stack y existen solo mientras la función está activa.

Memoria dinámica

C permite reservar memoria dinámicamente:

```
uint8_t *buffer = malloc(128);  
free(buffer);
```

Sin embargo, en muchos sistemas embebidos se evita el uso de memoria dinámica por razones de determinismo, fragmentación y seguridad.

14.6. Estructuras y memoria

Las estructuras permiten agrupar datos relacionados bajo un mismo tipo.

```
typedef struct {
    uint8_t hours;
    uint8_t minutes;
    uint8_t seconds;
} time_t;
```

Uso:

```
time_t current_time;

current_time.hours = 10;
current_time.minutes = 30;
current_time.seconds = 0;
```

Organización en memoria

Los campos de una estructura se almacenan en memoria en el orden en que se declaran. Sin embargo, el compilador puede insertar espacios adicionales llamados *padding* para cumplir restricciones de alineamiento.

Ejemplo:

```
typedef struct {
    uint8_t a;
    uint32_t b;
} example_t;
```

Aunque `a` ocupa 1 byte y `b` ocupa 4 bytes, la estructura puede ocupar más de 5 bytes debido al alineamiento.

Estructuras para periféricos

En muchos sistemas embebidos, los periféricos se controlan mediante direcciones de memoria. Dentro de un sistema, a cada periférico se le asigna una dirección específica en memoria.

Ejemplo

```
#define GPIO_BASE 0x40000000U
#define GPIO_OUT (*(volatile uint32_t*)(GPIO_BASE + 0x00))

GPIO_OUT = 1;
```

En sistemas embebidos, las estructuras pueden representar registros de hardware.

```
typedef struct {
    volatile uint32_t CTRL;
    volatile uint32_t STATUS;
```

```
volatile uint32_t DATA;
} uart_regs_t;
```

Luego se puede asociar la estructura a una dirección de memoria:

```
#define UART0_BASE 0x40000000U

#define UART0 ((uart_regs_t *) UART0_BASE)

UART0->CTRL = 1;
```

La palabra clave `volatile` indica al compilador que el valor puede cambiar fuera del flujo normal del programa, por ejemplo, por acción del hardware. Esto ya que, en sistemas embebidos, el compilador puede optimizar el código asumiendo que las variables solo cambian dentro del flujo del programa. Sin embargo, esto no siempre es cierto. Un ejemplo donde debe usarse `volatile` es en:

- registros de hardware
- variables modificadas por interrupciones
- comunicación con periféricos

Ejemplo

```
volatile uint32_t status;

while (status == 0) {
    // espera activa
}
```

Sin `volatile`, el compilador podría optimizar el código y convertirlo en un ciclo infinito.

Cualificadores de almacenamiento

En C, varias palabras clave controlan el comportamiento de las variables.

`const`

Indica que una variable no puede modificarse:

```
const uint32_t baudrate = 115200;
```

static

Permite que una variable mantenga su valor entre llamadas:

```
void counter(void) {
    static int count = 0;
    count++;
}
```

volatile

Indica que la variable puede cambiar externamente.

Resumen

Palabra clave	Propósito
const	Solo lectura
static	Persistencia local
volatile	Cambio externo

14.7. Punteros

Un puntero es una variable que almacena una dirección de memoria.

```
uint32_t value = 10;
uint32_t *ptr = &value;
```

En este ejemplo:

- `value` contiene el dato
- `&value` obtiene la dirección de memoria
- `ptr` almacena esa dirección
- `*ptr` accede al contenido apuntado

Acceso mediante punteros

```
*ptr = 20;
```

Esto modifica el valor de `value`.

Punteros y arreglos

En C, los arreglos están estrechamente relacionados con punteros.

```
uint8_t buffer[4] = {1, 2, 3, 4};

uint8_t first = buffer[0];
uint8_t also_first = *buffer;
```

Aritmética de punteros

```
uint8_t *p = buffer;

p++;
```

Después de `p++`, el puntero apunta al siguiente elemento del arreglo.

Punteros a estructuras

```
time_t now;
time_t *p_now = &now;

p_now->hours = 12;
```

El operador `->` se utiliza para acceder a campos de una estructura mediante un puntero.

14.8. Punteros a funciones

Un puntero a función almacena la dirección de una función. Esto permite seleccionar dinámicamente qué función ejecutar.

Ejemplo básico

```
void led_on(void)
{
    // Encender LED
}

void led_off(void)
{
    // Apagar LED
}

void (*action)(void);

action = led_on;
action();
```

Uso en sistemas embebidos

Los punteros a funciones son útiles para:

- tablas de interrupciones
- máquinas de estados
- callbacks
- drivers reutilizables

Ejemplo: tabla de estados

```
void state_idle(void);
void state_run(void);
void state_error(void);

void (*state_table[])(void) = {
    state_idle,
    state_run,
    state_error
};

uint8_t state = 1;

state_table[state]();
```

Este patrón permite implementar máquinas de estados de forma compacta y flexible.

14.9. Laboratorio: Sistema embebido interactivo con Arduino

En este laboratorio, se debe desarrollar un sistema embebido en Arduino que permita:

- leer entradas físicas (botón)
- generar salidas (LED)
- manejar interrupciones
- utilizar estructuras de datos
- aplicar punteros y manipulación de bits

Este laboratorio integra los conceptos fundamentales del lenguaje C aplicados a sistemas embebidos.

Materiales

- Arduino (UNO o similar)
- 1 LED
- 1 resistencia (220 Ω)
- 1 botón
- Protoboard y cables

Descripción del sistema

El sistema debe:

- alternar el estado de un LED cada vez que se presiona un botón
- utilizar una interrupción para detectar el evento
- mantener un contador de eventos
- estructurar los datos usando `struct`

Conexiones

- LED conectado al pin 13
- Botón conectado al pin 2 (interrupción externa)

Código base

```
#include <Arduino.h>

// Definicion de estructura
typedef struct {
    uint32_t count;
    uint8_t led_state;
} system_state_t;

// Variable global (compartida con ISR)
volatile uint8_t event_flag = 0;

// Estado del sistema
system_state_t state;

// Puntero al estado
system_state_t *p_state = &state;
```

```

// ISR (interrupcion)
void button_isr() {
    event_flag = 1;
}

void setup() {
    pinMode(13, OUTPUT);
    pinMode(2, INPUT_PULLUP);

    attachInterrupt(digitalPinToInterrupt(2), button_isr, FALLING);

    p_state->count = 0;
    p_state->led_state = 0;
}

void loop() {

    // Polling del flag generado por la ISR
    if (event_flag) {
        event_flag = 0;

        // Actualizar estado
        p_state->count++;

        // Toggle LED
        p_state->led_state ^= 1;

        digitalWrite(13, p_state->led_state);
    }
}

```

En el código anterior, identifique cómo se aplican los conceptos anteriores:

- **Interrupciones:** uso de `attachInterrupt` y de punteros a funciones.
- **volatile:** variable compartida con ISR
- **struct:** agrupación de datos del sistema
- **punteros:** acceso al estado mediante puntero
- **bitwise:** uso de XOR para alternar estado
- **flujo de programa:** loop principal con polling

Ahora, se le solicita extender el programa anterior con las siguientes funcionalidades:

- Agregar debounce por software
- Mostrar el contador por Serial
- Implementar el encendido o apagado de un LED mediante punteros a funciones.

Ejemplo: punteros a función

```
void led_on() { digitalWrite(13, HIGH); }  
void led_off() { digitalWrite(13, LOW); }  
  
void (*actions[2])() = {led_off, led_on};  
  
actions[p_state->led_state]();
```

Preguntas de análisis

1. ¿Por qué es necesario usar `volatile` en `event_flag`?
2. ¿Qué ocurre si se elimina la interrupción y se usa polling?
3. ¿Qué ventaja tiene usar una estructura para el estado del sistema?
4. ¿Qué sucede si el botón genera rebotes?
5. ¿Cómo cambiaría el sistema si se usa un temporizador en lugar de un botón?